

行动胜于言辞：面向生成式推荐的万亿参数序列转导器

Jiaqi Zhai¹ Lucy Liao¹ Xing Liu¹ Yueming Wang¹ Rui Li¹
Xuan Cao¹ Leon Gao¹ Zhaojie Gong¹ Fangda Gu¹ Michael He¹ Yinghai Lu¹ Yu Shi¹

Abstract

大规模推荐系统依赖高基数的异构特征，并且每天需要处理数百亿次用户行为。尽管使用包含数千种特征的海量数据进行训练，业界大多数深度学习推荐模型 (DLRM) 仍无法随算力增加而有效扩展。受 Transformer 在语言和视觉领域取得成功的启发，我们重新审视推荐系统中的基础设计选择。我们在生成式建模框架（“生成式推荐器”）下，将推荐问题重新表述为序列转导任务，并提出面向高基数、非平稳流式推荐数据的新架构 HSTU。在合成数据集和公开数据集上，HSTU 的 NDCG 相较基线最高提升 65.8%，且在长度为 8192 的序列上，比基于 FlashAttention2 的 Transformer 快 5.3 倍至 15.2 倍。基于 HSTU、拥有 1.5 万亿参数的生成式推荐器在在线 A/B 测试中将指标提升了 12.4%，并已部署于某大型互联网平台的多个业务场景，服务数十亿用户。更重要的是，生成式推荐器的模型质量在三个数量级的训练算力范围内，经验上呈幂律扩展，最高达到 GPT-3/LLaMa-2 的规模；这降低了未来模型开发所需的碳足迹，并进一步为推荐领域首批基础模型铺平了道路。

¹MRS, Meta AI. Correspondence to: <{jiaqiz, lucyyi, xingl, yuemingw, ruili}@meta.com>. Code available at <https://github.com/facebookresearch/generative-recommenders>.

1. 引言

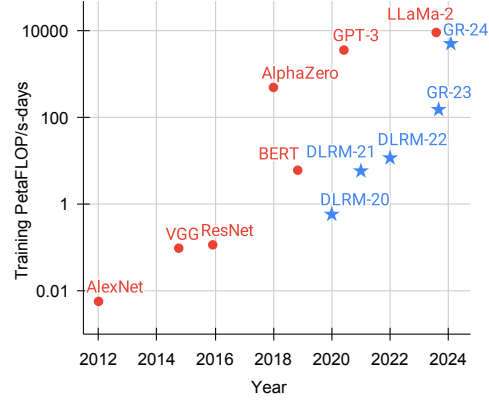


Figure 1. 历年来训练深度学习模型所使用的总算力。DLRM 结果来自 (Mudigere et al., 2022); GR 为本文已部署的模型。DLRM/GR 均在流式设定下持续训练；我们报告每年使用的算力。

推荐系统在在线内容平台和电子商务领域至关重要，每天都在为数十亿用户提供个性化体验。推荐领域的最先进方法一直以深度学习推荐模型 (DLRM) (Mudigere et al., 2022) 为基础，至今已有约十年 (Covington et al., 2016; Cheng et al., 2016; Zhou et al., 2018; Tang et al., 2020; Wang et al., 2021; Xia et al., 2023)。DLRM 的特点是使用异构特征，例如计数器和比率等数值特征、嵌入，以及创作者 ID、用户 ID 等类别特征。由于每分钟都会新增内容和产品，特征空间的基数极高，通常达到数十亿 (Ek-sombatchai et al., 2018)。为了利用数以万计的此类特征，DLRM 采用各种神经网络来组合特征、变换中间表示，并生成最终输出。

尽管使用了大量人工设计的特征集并基于海量数据训练，业界大多数 DLRM 仍无法随算力增加而良好扩展 (Zhao et al., 2023)。这一局限值得关注，且仍未得到解决。

受 Transformer 在语言和视觉领域取得成功的启发，

我们重新审视现代推荐系统中的基础设计选择。我们观察到，面向十亿用户规模的替代表述需要克服三项挑战。首先，推荐系统中的特征缺乏显式结构。尽管小规模设定中已经探索过序列表述（详见 Appendix B），包括高基数 ID、交叉特征、计数器、比率等在内的异构特征，在工业规模 DLRM 中起着关键作用 (Mudigere et al., 2022)。其次，推荐系统使用持续变化的十亿规模词表。与语言领域 100K 规模的静态词表 (Brown et al., 2020) 相比，十亿规模的动态词表带来了训练挑战；同时，由于需要以目标感知方式考虑数以万计的候选，还导致了高昂的推理成本 (Zhou et al., 2018; Wang et al., 2020)。最后，计算成本是实现大规模序列模型的主要瓶颈。GPT-3 使用数千块 GPU，在 1-2 个月内共训练了 300B 个 token (Brown et al., 2020)。这一规模看似惊人，但与用户行为的规模相比便不再如此。最大的互联网平台每天服务数十亿活跃用户，他们每天与数十亿条帖子、图片和视频交互。用户序列长度最高可达 10^5 (Chang et al., 2023)。因此，推荐系统每天需要处理的 token 数量，比语言模型在 1-2 个月内处理的数量高出数个数量级。

在本工作中，我们将用户行为视为生成式建模中的一种新模式。我们的关键洞见是：a) 只要采用适当的新特征空间，工业规模推荐器中的核心排序和检索任务就可表述为生成式建模问题；b) 这一范式使我们能够系统利用特征、训练和推理中的冗余来提高效率。得益于新的表述，我们部署的模型在计算复杂度上比此前最先进方法高出三个数量级，同时将顶线指标提升了 12.4%，如 Figure 1 所示。

我们的贡献如下。首先，我们提出了生成式推荐器 (GR)，见 Section 2；这是一种取代 DLRM 的新范式。我们将 DLRM 中的异质特征空间序列化并统一起来；当序列长度趋于无穷时，新方法可以近似完整的 DLRM 特征空间。这使我们能够将主要推荐问题——排序和检索——重新建模为 GR 中的纯序列转导任务。重要的是，这进一步使模型训练能够以序列生成方式完成，从而允许我们在计算量相同的情况下使用高出若干数量级的数据进行训练。

接下来，我们解决训练和推理全过程中的计算成本挑战。我们提出一种新的序列转导架构——层次序列转导单元 (HSTU)。HSTU 针对大规模非平稳词汇总表修改了注意力机制，并利用推荐数据集的特性，相较于 FlashAttention2 的 Transformer，在长度为 8192 的序列上实现了 5.3x 至 15.2x 的加速。此外，通过一种利用微批处理完全均摊计算成本的新算法 M-FALCON (Section 3.4)，我们可以在实现 1.50x-2.99x 加速的同时，为复杂 285x 的 GR 模型提供服务，而且全部都只使用传统 DLRM 的相同推理预算。

最后，我们在 Section 4 中通过合成数据集、公开数据集，以及在一个拥有数十亿日活用户的大型互联网平台多个界面上的部署，验证了所提技术。据我们所知，我们的工作首次表明，在生成式设置 (GR) 中，基于纯序列转导的 HSTU 等架构在大规模工业设置中显著优于 DLRM。值得注意的是，我们不仅克服了传统 DLRM 中已知的扩展瓶颈，还成功表明缩放律 (Kaplan et al., 2020) 适用于推荐，展现出推荐系统的 ChatGPT 时刻之潜力。

2. 将推荐表述为序列转导任务：从 DLRM 到 GR

2.1. 统一 DLRM 中的异质特征空间

现代 DLRM 模型通常使用大量类别（“稀疏”）特征和数值（“稠密”）特征进行训练。在 GR 中，我们将这些特征整合并编码到单一的统一时间序列中，如 Figure 2 所示。

类别（“稀疏”）特征。此类特征的例子包括用户点赞的物品、用户关注的某个类别（例如户外）中的创作者、用户语言、用户加入的社区、发起请求的城市等。我们按如下方式将这些特征序列化。首先，我们通常通过合并表示用户交互过的物品的特征，选取最长的时间序列作为主时间序列。其余特征通常是随时间缓慢变化的时间序列，例如人口统计信息或关注的创作者。我们通过保留每个连续片段中最早的条目来压缩这些时间序列，随后将结果合并到主时间序列中。由于这些时间序列变化非常缓慢，该

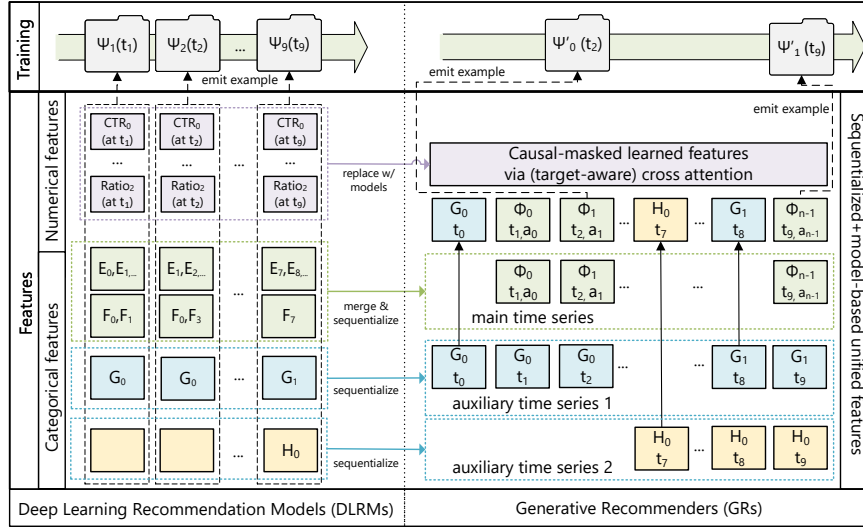


Figure 2. 特征与训练流程对比: DLRM 与 GR。 E, F, G, H 表示类别特征。 Φ_i 表示合并后主时间序列中的第 i 个物品。 $\Psi_k(t_j)$ 表示训练样本 k ，它在时间 t_j 产生。完整符号说明见 Appendix A。

方法不会显著增加总体序列长度。

数值 (“稠密”) 特征。 此类特征的例子包括加权且衰减的计数器、比率等。例如，某个特征可以表示用户过去对符合给定主题的物品的点击率 (CTR)。与类别特征相比，这些特征变化频繁得多，可能在每次 (用户，物品) 交互时都会变化。因此，从计算和存储角度看，完全序列化此类特征并不可行。然而，一个重要观察是，我们执行这些聚合所依据的类别特征 (例如物品主题、位置) 已经在 GR 中被序列化并编码。因此，在采用具有足够表达能力的序列转导架构并结合目标感知建模 (Zhou et al., 2018) 时，我们可以从 GR 中移除数值特征；随着 GR 的总体序列长度和计算量增加，该架构能够有效捕获数值特征。

2.2. 将排序与检索重新建模为序列转导任务

给定按时间顺序排列的 n 个 token 的列表 $x_0, x_1, \dots, x_{n-1}$ ($x_i \in \mathbb{X}$) 及这些 token 被观测到的时间 $t_0, t_1, \dots, t_{n-1}$ ，序列转导任务将该输入序列映射为输出 token $y_0, y_1, \dots, y_{n-1}$ ($y_i \in \mathbb{X} \cup \{\emptyset\}$)，其中 $y_i = \emptyset$ 表示 y_i 未定义。

我们用 $\Phi_i \in \mathbb{X}_c$ ($\mathbb{X}_c \subseteq \mathbb{X}$) 表示系统提供给用户的一项内容 (例如图像或视频)。由于新内容不断产生， $\mathbb{X}_c$ 和 $\mathbb{X}$ 是非平稳的。用户可以响应 Φ_i ，采取某个动作 a_i (例如点赞、跳过、看完视频并分享) $a_i \in \mathbb{X}$ 。

我们用 n_c 表示用户交互过的内容总数。

随后，在因果自回归设置下，可将标准排序和召回任务定义为序列转导任务 (Table 1)。我们有如下观察：

任务	规范 (输入 / 输出)
排序	$x_i^S \quad \Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-1}, a_{n_c-1}$
	$y_i^S \quad a_0, \emptyset, a_1, \emptyset, \dots, a_{n_c-1}, \emptyset$
召回	$x_i^S \quad (\Phi_0, a_0), (\Phi_1, a_1), \dots, (\Phi_{n_c-1}, a_{n_c-1})$
	$y_i^S \quad \Phi'_1, \Phi'_2, \dots, \Phi'_{n_c-1}, \emptyset$ (取 $\Phi'_i = \Phi_i$ ，若 a_i 为正向；否则取 $\emptyset$)

Table 1. 将排序和召回表述为序列转导任务。为简洁起见，省略其他类别特征。我们在 Appendix B.2 中将 GR 与传统序列推荐器进行比较。

检索。 在推荐系统的检索阶段，我们学习分布 $p(\Phi_{i+1}|u_i)$ ，其定义在 $\Phi_{i+1} \in \mathbb{X}_c$ 上，其中 u_i 是用户在 token i 处的表示。典型目标是选择 $\arg \max_{\Phi \in \mathbb{X}_c} p(\Phi|u_i)$ 以最大化某种奖励。这与标准自回归设置有两点不同。首先， x_i 的监督信号 y_i 不一定是 Φ_{i+1} ，因为用户可能对 Φ_{i+1} 作出负面反应。其次， y_i 在 x_{i+1} 表示与交互无关的类别特征 (例如人口统计信息) 时未定义。

排序。 GR 中的排序任务带来了独特挑战，因为工业推荐系统通常需要“目标感知”形式化。在此类设置中，目标 Φ_{i+1} 与历史特征之间的“交互”需尽早发生，而在标准自回归设置中，“交互”发生得较晚 (例如在编码器输出之后通过 softmax)，因而

无法实现。我们通过交错排列物品和操作来解决这一问题，如 Table 1 所示，这使排序任务可以表示为 $p(a_{i+1}|\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{i+1})$ （位于类别特征之前）。实际中，我们应用一个小型神经网络，将 Φ_{i+1} 处的输出转换为多任务预测。重要的是，这使我们能够一次性对所有 n_c 次交互应用目标感知的交叉注意力。

2.3. 生成式训练

工业推荐器通常在流式设置下训练，其中每个样本都会在可用时按顺序处理。在该设置下，基于自注意力的序列转导架构（如 Transformer (Vaswani et al., 2017)）的总计算需求按 $\sum_i n_i(n_i^2 d + n_i d_{ff} d)$ 缩放，其中 n_i 是用户 i 的 token 数量， d 是嵌入维度。括号中的第一部分来自自注意力；我们假定其缩放因子为 $O(n^2)$ ，因为多数二次算法都会在质量上作出权衡，且在实际运行时间上逊于二次算法 (Dao et al., 2022)。第二部分来自逐点 MLP 层，其隐藏层大小为 $O(d_{ff}) = O(d)$ 。取 $N = \max_i n_i$ 后，总体时间复杂度降为 $O(N^3 d + N^2 d^2)$ ，但对于推荐设置而言仍成本高昂。

为以可扩展方式解决长序列上的序列转导模型训练难题，我们从传统的曝光级训练转向生成式训练，将计算复杂度降低 $O(N)$ 倍，如图 2 顶部所示。这样，编码器成本便可由多个目标分摊。更具体地，当我们采样第 i 个用户且采样率为 $s_u(n_i)$ 时，总训练成本现按 $\sum_i s_u(n_i) n_i(n_i^2 d + n_i d^2)$ 缩放，并可降至 $O(N^2 d + N d^2)$ ，只需将 $s_u(n_i)$ 设为 $1/n_i$ 。在工业级系统中实现这种采样的一种方式，是在用户请求或会话结束时生成训练样本，从而得到 $\hat{s}_u(n_i) \propto 1/n_i$ 。

3. 用于生成式推荐的高性能自注意力编码器

为了将 GR 扩展至具有大规模非平稳词表的工业级推荐系统，我们接下来引入一种新的编码器设计——分层序列转导单元（HSTU）。HSTU 由多个通过残差连接 (He et al., 2015) 相连的相同层堆叠而成。每层包含三个

$$A(X)V(X) = \phi_2 \left(Q(X)K(X)^T + \text{rab}^{p,t} \right) V(X) \quad (2)$$

$$Y(X) = f_2(\text{Norm}(A(X)V(X)) \odot U(X)) \quad (3)$$

其中， $f_i(X)$ 表示 MLP；我们使用一个线性层，并使用 $f_i(X) = W_i(X) + b_i$ 处理 f_1 和 f_2 以降低计算复杂度，同时利用融合核进一步批处理查询 $Q(X)$ 、键 $K(X)$ 、值 $V(X)$ 和门控权重 $U(X)$ 的计算； ϕ_1 和 ϕ_2 表示非线性函数，二者均采用 SiLU (Elfwing et al., 2017)；Norm 为层归一化； $\text{rab}^{p,t}$ 表示相对注意力偏置 (Raffel et al., 2020)，其中融合了位置 (p) 和时间 (t) 信息。所用完整符号见 Table 9。

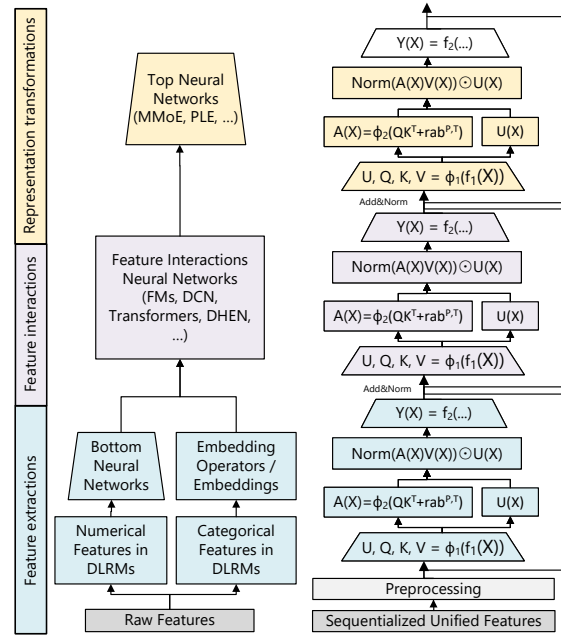


Figure 3. 关键模型组件比较：DLRM 与 GR。左侧展示完整的 DLRM 设置 (Mudigere et al., 2022)，右侧展示简化的 HSTU。

HSTU 编码器设计允许用单一模块化块替换 DLRM 中的异质模块。我们观察到，DLRM 实际上包含三个主要阶段：特征提取、特征交互和表示转换。特征提取获取类别特征的池化嵌入表示。其最先进的版本可概括为成对注意力和目标感知池化 (Zhou et al., 2018)，这些均可由 HSTU 层捕获。

特征交互是 DLRM 最关键的部分。常用方法包括因子分解机及其神经网络变体 (Rendle, 2010; Guo et al., 2017; Xiao et al., 2017)、高阶特征交互 (Wang et al., 2021) 等。HSTU 通过允许注意力池化特征经由 $\text{Norm}(A(X)V(X)) \odot U(X)$ 直接与其他特征

$$U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X))) \quad (1)$$

架构	HR@10	HR@50
Transformers	.0442	.2025
HSTU (-rab ^{p,t} , Softmax)	.0617	.2496
HSTU (-rab ^{p,t})	.0893	.3170

Table 2. 单遍流式设置下的合成数据。

“交互”，从而替代特征交互。这一设计的动机源于使用学习得到的 MLP 近似点积存在困难 (Rendle et al., 2020; Zhai et al., 2023a)。由于 SiLU 应用于 $U(X)$, $\text{Norm}(A(X)V(X)) \odot U(X)$ 也可视为 SwiGLU (Shazeer, 2020) 的一种变体。

表示转换通常通过专家混合 (MoE) 和路由来处理多样、异质的人群。一个关键思想是通过针对不同用户专门化子网络来执行条件计算 (Ma et al., 2018; Tang et al., 2020)。在归一化因子的意义下, HSTU 中的逐元素点积实际上可以执行 MoE 中使用的门控操作。

3.1. 逐点聚合注意力

HSTU 采用一种新的逐点聚合 (归一化) 注意力机制 (相比之下, softmax 注意力在整个序列上计算归一化因子)。其动机来自两个方面。第一, 与目标相关的先前数据点数量, 是反映用户偏好强度的一个强特征, 而经过 softmax 归一化后很难捕获这一特征。这一点至关重要, 因为我们既需要预测交互强度 (例如在给定物品上的停留时间), 也需要预测物品的相对顺序 (例如预测一个排序以最大化 AUC)。第二, 尽管 softmax 激活在设计上对噪声具有鲁棒性, 但它不太适合流式设置中的非平稳词表。

所提出的逐点聚合注意力机制如 Equation (2) 所示。重要的是, 逐点池化后需要层归一化来稳定训练。理解这一设计的一种方式, 是使用遵循 Dirichlet Process 的合成数据, 该过程在非平稳词汇表上生成流式数据 (详见附录 C)。在这种设置下, 我们可以观察到 softmax 和逐点注意力设置之间高达 44.7% 的差距, 如 Table 2 所示。

3.2. 利用稀疏性并通过算法提高稀疏性

在推荐系统中, 用户历史序列的长度往往遵循偏态分布, 导致输入序列稀疏, 尤其是在超长序列设置

中。可以利用这种稀疏性显著提高编码器效率。为此, 我们为 GPU 开发了一种高效注意力内核, 其以类似于 (Rabe & Staats, 2021; Dao et al., 2022) 的方式融合连续的 GEMM, 但执行完全不规则化的注意力计算。这实质上将注意力计算转换为不同大小的分组 GEMM (Appendix G)。因此, HSTU 中的自注意力变得受内存带宽限制, 且其内存访问量按 $\Theta(\sum_i n_i^2 d_{qk}^2 R^{-1})$ 扩展, 其中 n_i 是样本 i 的序列长度, d_{qk} 是注意力维度, R 是寄存器大小。该方法本身即可带来 2-5x 的吞吐量提升, 如 Section 4.2 所述。

我们进一步通过算法方式, 利用随机长度 (SL) 提高用户历史序列的稀疏性。推荐中的用户历史序列有一项关键特征: 用户行为具有时间重复性, 因为用户行为会在其整个交互历史中的多个尺度上显现。这为在不损害模型质量的情况下人为提高稀疏性提供了机会, 从而显著降低按 $\Theta(\sum_i n_i^2)$ 扩展的编码器成本。

我们可以将用户 j 的历史表示为序列 $(x_i)_{i=0}^{n_{c,j}}$, 其中 $n_{c,j}$ 是用户交互过的内容数量。令 $N_c = \max_j n_{c,j}$ 。令 $(x_{i_k})_{k=0}^L$ 为长度为 L 、从原始序列 $(x_i)_{i=0}^{n_{c,j}}$ 构造的子序列。SL 按如下方式选择输入序列:

$$\begin{aligned} & (x_i)_{i=0}^{n_{c,j}} \text{ 若 } n_{c,j} \leq N_c^{\alpha/2} \\ & (x_{i_k})_{k=0}^{N_c^{\alpha/2}} \text{ 若 } n_{c,j} > N_c^{\alpha/2}, \text{ 概率为 } 1 - N_c^{\alpha}/n_{c,j}^2 \\ & (x_i)_{i=0}^{n_{c,j}} \text{ 若 } n_{c,j} > N_c^{\alpha/2}, \text{ 概率为 } N_c^{\alpha}/n_{c,j}^2 \end{aligned} \quad (4)$$

这将与注意力相关的复杂度降至 $O(N_c^{\alpha} d) = O(N^{\alpha} d)$, 其中 $\alpha \in (1, 2]$ 。更全面的子序列选择讨论见 Appendix F.1。我们注意到, 将 SL 应用于训练能够形成具有成本效益的系统设计, 因为训练的计算成本通常显著高于推理。

Table 3 给出了不同序列长度和 α 值对应的稀疏度 (见 Appendix F), 其采用一个具有 30 天用户历史的代表性工业规模配置。对模型质量导致可忽略回归的设置以下划线和蓝色突出显示。标记为 “ $\alpha = 2.0$ ” 的行表示未应用 SL 时的基准稀疏度情形。较小的 α 可适用于更长的序列, 直至我们测试的最长序列长度 8,192。

Alpha (α)	最大序列长度			
	1,024	2,048	4,096	8,192
1.6	71.5%	76.1%	80.5%	<u>84.4%</u>
1.7	<u>56.1%</u>	<u>63.6%</u>	<u>69.8%</u>	<u>75.6%</u>
1.8	<u>40.2%</u>	<u>45.3%</u>	<u>54.1%</u>	<u>66.4%</u>
1.9	<u>17.2%</u>	<u>21.0%</u>	<u>36.3%</u>	<u>64.1%</u>
2.0	<u>3.1%</u>	<u>6.6%</u>	<u>29.1%</u>	<u>64.1%</u>

Table 3. 随机长度 (SL) 对序列稀疏度的影响。

3.3. 最小化激活内存用量

在推荐系统中, 使用大批次对于训练吞吐量 (Mudigere et al., 2022) 和模型质量 (Yang et al., 2020; Chen et al., 2020; Zhai et al., 2023a) 都至关重要。因此, 激活内存用量成为主要的扩展瓶颈; 这与通常采用小批次训练、内存占用主要由参数主导的大语言模型不同。

与 Transformer 相比, HSTU 采用简化且完全融合的设计, 显著降低了激活内存用量。首先, HSTU 将注意力之外的线性层数量从六层减少到两层, 这与近期使用逐元素门控来减少 MLP 计算的工作一致 (Hua et al., 2022; Gu et al., 2022)。其次, HSTU 积极地将计算融合为单个算子, 包括 $\phi_1(f_1(\cdot))$ (见 Equation (1)), 以及 Equation (3) 中的层归一化、可选 dropout 和输出 MLP。这种简化设计将 bfloat16 下每层的激活内存用量降至 $2d + 2d + 4hd_{qk} + 4hd_v + 2hd_v = 14d$

作为对比, Transformer 在注意力之后使用前馈层和 dropout (中间状态为 $3hd_v$), 随后是由层归一化、线性层、激活、线性层和 dropout 构成的逐点前馈块, 其中间状态为 $2d + 4d_{ff} + 2d + 1d = 4d + 4d_{ff}$ 。这里, 我们作出标准假设: $hd_v \geq d$ 且 $d_{ff} = 4d$ (Vaswani et al., 2017; Brown et al., 2020)。因此, 在计入输入和输入层归一化 ($4d$) 以及 qkv 投影后, 激活状态总量为 $33d$ 。由此, HSTU 的设计能够扩展到深度为 $> 2x$ 的网络。

此外, 用于表示词表的大规模原子 ID 也需要大量内存。若词表规模为 10b、嵌入维度为 512d, 并使用 Adam 优化器, 仅以 fp32 存储嵌入和优化器状态就需要 60TB 内存。为缓解内存压力, 我们采用逐行 AdamW 优化器 (Gupta et al., 2014; Khudia

et al., 2021), 并将优化器状态存放在 DRAM 上, 使 HBM 中每个浮点数的占用从 12 字节降至 2 字节。

3.4. 通过成本摊销扩展推理

我们要解决的最后一个挑战, 是推荐系统在服务时需要处理的大量候选项。我们聚焦于排序, 因为对于检索, 编码器成本可以完全均摊, 而且无论是利用量化、哈希或分区的 MIPS (Jegou et al., 2011; Shrivastava & Li, 2014; Li et al., 2002; Zhai et al., 2011), 还是通过束搜索或层次检索的非 MIPS 情形 (Zhuo et al., 2020; Zhai et al., 2023a), 都已有高效算法。

对于排序, 我们最多有数万个候选项 (Covington et al., 2016; Wang et al., 2020)。我们提出一种算法 M-FALCON (Microbatched-Fast Attention Leveraging Cacheable Operations) 用于对 m 个候选项执行推理, 其输入序列大小为 n 。

在一次前向传播中, M-FALCON 并行处理 b_m 个候选项, 方法是修改注意力掩码和 $\text{rab}^{p,t}$ 偏置, 使得对 b_m 个候选项执行的注意力操作完全相同。这将应用交叉注意力的成本从 $O(b_m n^2 d)$ 降至 $O((n + b_m)^2 d) = O(n^2 d)$, 前提是 b_m 相对于 n 可视为小常数。我们可选地将全部 m 个候选项分成 $\lceil m/b_m \rceil$ 个大小为 b_m 的微批, 以利用编码器级 KV 缓存 (Pope et al., 2022): 可以跨前向传播降低成本, 也可跨请求降低尾延迟 (更详细的讨论见 Appendix H)。

总体而言, M-FALCON 使模型复杂度能够随传统 DLRM 排序阶段中的候选项数量线性扩展; 我们成功在典型排序配置的固定推理预算下, 以 1.5x-3x 的吞吐量应用了复杂 285x 的目标感知交叉注意力模型, 如 Section 4.3 所述。

4. 实验

4.1. 验证 HSTU 编码器的归纳假设

4.1.1. 传统序列设定

我们首先在两个流行的推荐数据集 MovieLens 和 Amazon Reviews 上评估 HSTU 的性能。我们遵循

Table 4. 多遍历、全量打乱设置下各方法在公开数据集上的评估。

	方法	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2853	.5474	.7528	.1603	.2498
	HSTU	.3097 (+8.6%)	.5754 (+5.1%)	.7716 (+2.5%)	.1720 (+7.3%)	.2606 (+4.3%)
	HSTU-large	.3294 (+15.5%)	.5935 (+8.4%)	.7839 (+4.1%)	.1893 (+18.1%)	.2771 (+10.9%)
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	.3567 (+22.8%)	.6149 (+11.8%)	.8076 (+5.5%)	.2106 (+30.0%)	.2971 (+17.9%)
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	.0469 (+60.6%)	.1066 (+46.2%)	.1876 (+33.9%)	.0257 (+65.8%)	.0508 (+45.1%)

文献中的序列推荐设定，包括全量打乱和多轮训练。基线采用 SASRec，即一种最先进的 Transformer 实现 (Kang & McAuley, 2018)¹。与近期工作 (Dallmann et al., 2021; Zhai et al., 2023a) 一致，我们在整个语料库上报告 Hit Rate@K 和 NDCG@K。

结果见 Table 4。“SASRec (2023)”表示 (Zhai et al., 2023a) 中报告的最佳 SASRec 配置。标为“HSTU”的行使用与 SASRec 相同的配置（相同的层数、头数等）。“HSTU-large”表示更大的 HSTU 编码器（层数为 4 倍、头数为 2 倍）。结果表明：a) 经过面向推荐优化的设计，HSTU 在采用相同配置时显著优于基线；b) HSTU 在扩大规模后，性能进一步提升。

需要指出的是，此处使用的评估方法与工业规模设定有显著差异，因为全量打乱和多轮训练通常不适用于工业界使用的流式设定 (Liu et al., 2022)。

4.1.2. 工业规模流式设置

接下来，我们使用工业规模数据集，在流式设置中比较 HSTU、消融后的 HSTU 和 Transformer 的性能。在本节余下部分中，我们报告排序的归一化熵 (NE) (He et al., 2014)。我们在 100B 个样本 (DLRM 等价) 上训练模型，每个作业使用 64-256 个 H100。由于排序在多任务设置中完成，我们报告主要交互事件 (“E-Task”) 和主要消费事件 (“C-Task”)。在我们的语境中，NE 降低 0.001 被视为显著，因为它

¹其他基线的结果见 Appendix D。

Table 5. 单遍流式设置下，HSTU、消融后的 HSTU 和 Transformer 在工业规模数据集上的评估。

架构	召回	排序 (NE)	
	log pplx.	E-Task	C-Task
Transformers	4.069	NaN	NaN
HSTU (-rab ^{p,t} , Softmax)	4.024	.5067	.7931
HSTU (-rab ^{p,t})	4.021	.4980	.7860
Transformer++	4.015	.4945	.7822
HSTU (original rab)	4.029	.4941	.7817
HSTU	3.978	.4937	.7805

通常会为数十亿用户带来 .5% 的顶层指标改善。对于检索，由于其设置与语言建模类似，我们报告对数困惑度。我们在较小规模设置中固定编码器参数（排序为 $l = 3$ 、 $n = 2048$ 、 $d = 512$ ，检索为 $l = 6$ 、 $n = 512$ 、 $d = 256$ ），并由于资源限制对其他超参数进行网格搜索。

结果见 Table 5。首先，HSTU 显著优于 Transformer，尤其是在排序任务中，这可能归功于逐点注意力和改进的相对注意力偏置。其次，消融版 HSTU 与 HSTU 之间的差距证实了我们设计的有效性。由于训练稳定性的原因，基于 Softmax 的 HSTU 和 Transformer 的最优学习率约比其余模型低 10x。即便采用更低的学习率和预归一化残差连接 (Xiong et al., 2020)，我们在排序任务中使用标准 Transformer 时仍频繁遇到损失爆炸最后，HSTU 优于 LLM 中常用的一种 Transformer 变体 Transformer++ (Touvron et al., 2023a)，后者使用 RoPE (Su et al., 2023)、SwiGLU 等。总体而言，在这一小规模设置中，HSTU 的质量更高，实际运行

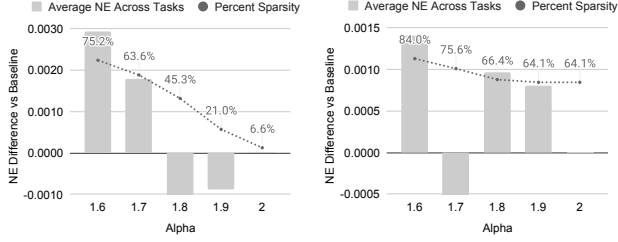


Figure 4. 随机长度 (SL) 对指标的影响。左: $n = 4096$ 。右: $n = 8192$ 。完整结果见 Appendix F。

速度快 1.5x–2x, 且 HBM 用量减少 50%。

4.2. 编码器效率

随机长度。 Figure 4 和 Figure 5 (a) 展示了随机长度 (SL) 对模型指标的影响。在 $\alpha = 1.6$ 时, 长度为 4096 的序列在大多数情况下会变为长度为 776 的序列, 即移除 80% 以上的 token。即使稀疏率提高到 64%–84%, 主要任务上得到的 NE 下降也不超过 0.002 (0.2%)。这些证据表明, 对于合适的 α , SL 不会对模型质量产生负面影响, 并允许实现的高稀疏性以降低训练成本。我们还在 Appendix F.3 中验证了 SL 显著优于现有的长度外推技术。

编码器效率。 Figure 5 比较了 HSTU 和 Transformer 编码器在训练与推理设置下的效率。对于 Transformer, 我们使用最先进的 FlashAttention-2 (Dao, 2023) 实现。我们考虑从 1,024 到 8,192 的序列长度, 并在训练期间应用 随机长度 (SL)。在评估中, 我们对 HSTU 和 Transformer 使用相同配置 ($d = 512$, $h = 8$, $d_{qk} = 64$), 并消融相对注意力偏置因为即使不使用 $\text{rab}^{p,t}$, HSTU 也优于 Transformer, 如 Section 4.1.2 所示。我们在 NVIDIA H100 GPU 上以 bfloat16 比较编码器级性能。总体而言, HSTU 在训练和推理中的效率分别最高可达 Transformer 的 15.2x 和 5.6x。

此外, 如第 3.3 节所述, 激活内存用量的降低使我们能够使用 HSTU 构建比 Transformer 深 2x 以上的网络。

4.3. 工业级流式设置中的生成式推荐器与 DLRM

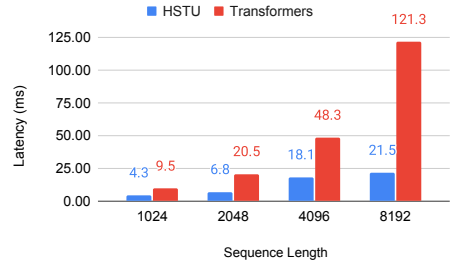
最后, 我们在工业级流式设置下, 将 GR 的端到端性能与最先进的 DLRM 基线进行比较。我们的 GR



(a) 训练 NE。



(b) 训练加速比。



(c) 推理加速比。

Figure 5. 编码器层面的效率: HSTU 与基于 FlashAttention2 的 Transformer 在训练 (a、b) 和推理 (c) 方面的比较。

实现反映了生产环境中使用的典型配置, 而 DLRM 设置则反映了数百人多年来的迭代成果。由于推荐系统的召回阶段会使用多个生成器, 我们同时报告新增 GR (“add source”) 和用 GR 替换现有主要 DLRM 来源 (“replace source”) 的在线结果。Table 6 和 Table 7 表明, GR 不仅在线下显著优于 DLRM, 还在 A/B 测试中带来了 12.4% 的增益。

如 Section 2 所述, GR 建立在原始类别交互特征之上, 而 DLRM 通常使用数量多得多的特征进行训练, 其中大多数是从原始信号中人工构造的。如果向 DLRM 提供与 GR 相同的特征集合 (“DLRM (abl. features)”), DLRM 的性能会显著下降, 这表

Table 6. 召回模型的离线/在线比较。

方法	离线 HR@K		在线指标	
	K=100	K=500	E-Task	C-Task
DLRM	29.0%	55.5%	+0%	+0%
DLRM (abl. features)	28.3%	54.3%	—	—
GR (content-based)	11.6%	18.8%	—	—
GR (interactions only)	35.6%	61.7%	—	—
GR (new source)	36.9%	62.4%	+6.2%	+5.0%
GR (replace source)			+5.1%	+1.9%

Table 7. 排序模型的离线/在线比较。

方法	离线 NE		在线指标	
	E-Task	C-Task	E-Task	C-Task
DLRM	.4982	.7842	+0%	+0%
DLRM (DIN+DCN)	.5053	.7899	—	—
DLRM (abl. features)	.5053	.7925	—	—
GR (interactions only)	.4851	.7903	—	—
GR	.4845	.7645	+12.4%	+4.4%

明 GR 能够通过其架构和统一特征空间有效捕获这些特征。

我们还在 Section 2.2 中, 通过将 GR 建模与仅考虑用户交互物品 (Kang & McAuley, 2018) 的传统序列推荐器设置 (“GR (interactions only)”) 进行比较, 进一步验证了 GR 建模。其结果显著更差: 在主要消费任务中, 其排序变体的 NE 比 GR 低 2.6%。

考虑到基于内容的方法 (包括 LM) 的流行, 我们还纳入了一个仅使用内容特征的 GR 基线 (“GR (content-based)”)。基于内容的基线与 DLRM/GR 之间巨大的性能差距, 凸显了高基数用户动作的重要性。

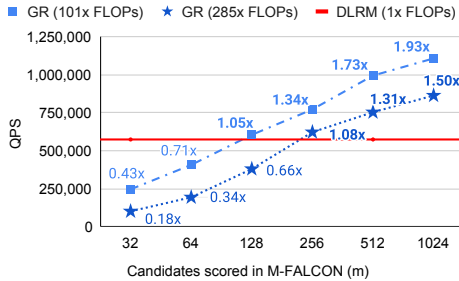


Figure 6. 在最具挑战性的排序设定下比较推理吞吐量。完整结果见 Appendix H.1。

最后, 我们在 Figure 6 中将 GR 的效率与生产 DLRM 进行了比较。尽管 GR 模型的计算复杂度高出 285x, 但在对 1024/16384 个候选进行打分

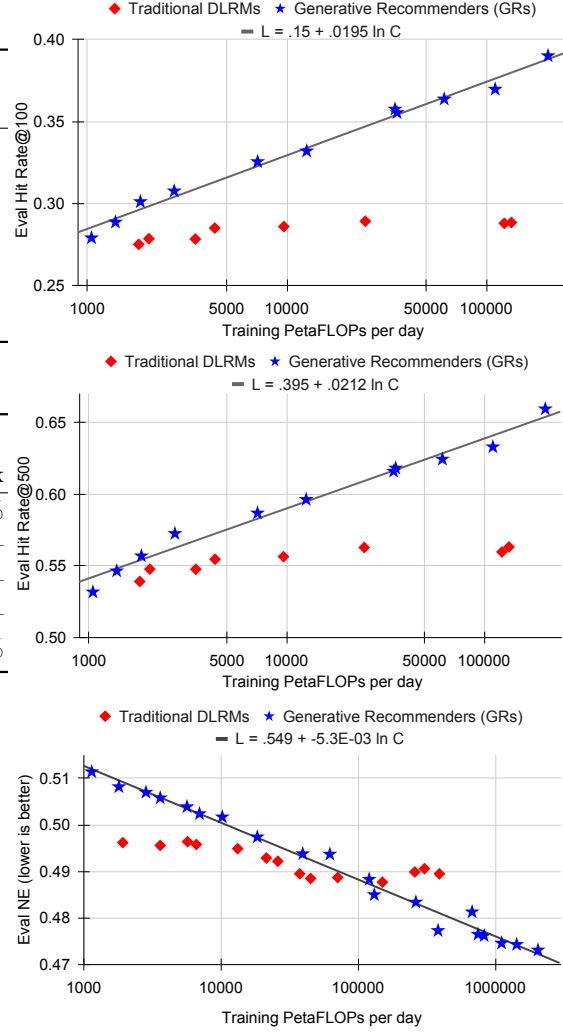


Figure 7. 可扩展性: 大规模工业设置中, DLRM 与 GR 在检索 (上、中) 和排序 (下) 任务上的对比。HR 提高 +0.005 和 NE 降低 -0.001 表示显著改进。

时, 我们实现了 1.50x/2.99x 更高的 QPS, 这归功于 HSTU 以及新颖的 M-FALCON 算法 (见 Section 3.4)。

4.3.1. 推荐系统的缩放定律

众所周知, 在大规模工业设置中, DLRM 的质量会在某些计算量和参数量区间趋于饱和 (Zhao et al., 2023)。我们比较 GR 和 DLRM 的可扩展性, 以更好地理解这一现象。

由于特征交互层对 DLRM 的性能至关重要 (Mudigere et al., 2022), 我们试验了 Transformer (Vaswani et al., 2017)、DHEN (Zhang et al., 2022), 以及 DCN 的一种变体 (Wang et al., 2021), 该变体加入了我们生产排序设置中使用的残差连接 (He et al., 2015),

以扩展 DLRM 基线。对于检索基线，由于基线采用残差设置，我们扩大了隐藏层大小、嵌入维度和层数。对于基于 HSTU 的生成式推荐器 (GR)，我们通过调整 HSTU 的超参数来扩展模型，包括残差层数、序列长度、嵌入维度、注意力头数等。我们还调整了检索中的负样本数量。

结果见 Figure 7。在低计算量区间，由于手工特征，DLRM 可能优于 GR，这印证了特征工程在传统 DLRM 中的重要性。然而，GR 相对 FLOPs 表现出显著更好的可扩展性，而 DLRM 的性能趋于平台，与先前工作的发现一致。我们还观察到，无论相对嵌入参数还是非嵌入参数，GR 的可扩展性都更好，可扩展到 1.5 万亿参数的模型，而 DLRM 的性能在约 2000 亿参数时饱和。

最后，在超参数适当时，我们所有主要指标，包括检索的 Hit Rate@100 和 Hit Rate@500，以及排序的 NE，在经验上均按所用计算量的幂律缩放。我们在三个数量级上观察到这一现象，直到能够测试的最大模型 (8,192 序列长度、1,024 嵌入维度、24 层 HSTU)；在该点，我们使用的总计算量（由于采用标准流式训练设置，按 365 天归一化）接近 GPT-3 (Brown et al., 2020) 和 LLaMa-2 (Touvron et al., 2023b) 的总训练计算量，如 Figure 1 所示。在合理范围内，与应用的训练总计算量相比，具体模型超参数的作用较小。与语言建模 (Kaplan et al., 2020) 不同，序列长度在 GR 中的作用显著更重要，因此必须同步扩展序列长度和其他参数。这也许是我们所提方法最重要的优势，因为我们首次表明 LLM 中的缩放律也可能适用于大规模推荐系统。

5. 相关工作

以往的序列推荐器工作将用户交互简化为单一的同质物品序列 (Hidasi et al., 2016; Kang & McAuley, 2018)。序列方法的工业规模应用主要采用成对注意力 (Zhou et al., 2018)，或将序列编码器作为 DLRM 的一部分 (Chen et al., 2019; Xia et al., 2023)。为提高效率，已有工作探索用多阶段注意力取代自注意力 (Chang et al., 2023)。检索领域也探索了将 ID 表示为 token 序列的生成式方法 (Zhuo et al., 2020)。

我们在 Appendix B.1 中对以往工作作了更全面的讨论。

高效注意力一直是重要研究方向等。近期也有工作探索序列转导设定的替代表述 (Gu et al., 2022; Hua et al., 2022)。HSTU 的逐元素门控设计尤其受到 FLASH (Hua et al., 2022) 的启发。近期的硬件感知表述已被证明可显著减少内存使用 (Rabe & Staats, 2021; Korthikanti et al., 2022; Zhai et al., 2023b)，并显著改善实际运行时间 (Dao et al., 2022)。长度外推使在较短序列上训练的模型具备泛化能力，但大多数工作侧重于微调或改进偏置机制 (Press et al., 2022)。与之不同，我们受深度维度随机性研究 (Huang et al., 2016) 启发，在长度维度引入随机性。

对大语言模型 (LLM) 的兴趣推动了将各种推荐任务作为上下文学习 (Sileo et al., 2022)、指令微调 (Bao et al., 2023)，或在预训练 LLM 上进行迁移学习 (Li et al., 2023) 的研究。LLM 中蕴含的世界知识可迁移到下游任务 (Cui et al., 2022)，并在零样本或少样本场景中改善推荐。用户行为序列的文本表示也在中等规模数据集上展现出良好的扩展特性 (Shin et al., 2023)。大多数面向推荐的 LLM 研究都聚焦于低数据量场景；在大规模设定中，它们尚未超越协同过滤

6. 结论

我们提出了生成式推荐器 (GR)，这是一种将排序和召回表述为序列转导任务、从而能够以生成方式进行训练的新范式。这得益于新颖的 HSTU 编码器设计，在长度为 8192 的序列上，其速度比最先进的 Transformer 快 5.3x–15.2x，以及 M-FALCON 等新型训练和推理算法。借助 GR，我们部署的模型复杂度达到原来的 285x，同时推理计算量反而更少。GR 和 HSTU 在生产环境中带来了 12.4% 的指标提升，并展现出优于传统 DLRM 的缩放性能。我们的结果证实，用户动作是生成建模中一种尚未得到充分探索的模式——正如标题所言，“行动胜于言语”。

本工作对特征的大幅简化使跨领域采用统一特征空

间成为可能，从而为首批面向推荐、搜索和广告的基础模型铺平了道路。GR 的完全序列化设置还使推荐能够被表述为端到端的生成式任务。这两点共同使推荐系统能够更全面地帮助用户。

影响声明

我们相信，本工作具有广泛的积极影响。减少推荐、搜索和广告系统对大量异构特征的依赖，可以在改善用户体验的同时，使这些系统更加保护隐私。通过完全序列化的表述，使推荐系统能够将用户的长期结果归因于短期决策，有望减少互联网上不符合用户长期目标的内容（包括标题党和虚假新闻），并使平台激励机制与用户价值观更好地保持一致。最后，基础模型和扩展定律的应用有助于减少推荐、搜索及相关用例所需模型研究与开发产生的碳足迹。

致谢

本工作汇聚了数百人的共同努力，没有以下贡献者的工作就不可能完成（按字母顺序排列）：Adnan Akhundov, Bugra Akyildiz, Shabab Ayub, Alex Bao, Renqin Cai, Jennifer Cao, Guoqiang Jerry Chen, Lei Chen, Sean Chen, Xianjie Chen, Huihui Cheng, Weiwei Chu, Ted Cui, Shiyan Deng, Nimit Desai, Fei Ding, Francois Fagan, Lu Fang, Liang Guo, Liz Guo, Jeevan Gyawali, Yuchen Hao, Daisy Shi He, Samuel Hsia, Jie Hua, Yanzun Huang, Hongyi Jia, Rui Jian, Jian Jin, Rahul Kindi, Changkyu Kim, Yejin Lee, Fu Li, Hong Li, Shen Li, Wei Li, Zhijing Li, Xueting Liao, Emma Lin, Hao Lin, Jingzhou Liu, Xingyu Liu, Kai Londen-berg, Liang Luo, Linjian Ma, Matt Ma, Yun Mao, Bert Maher, Matthew Murphy, Satish Nadathur, Min Ni, Jongsoo Park, Jing Qian, Lijing Qin, Alex Singh, Timothy Shi, Dennis van der Staay, Xiao Sun, Colin Taylor, Shin-Yeh Tsai, Rohan Varma, Omkar Vichare, Alyssa Wang, Pengchao Wang, Shengzhi Wang, Wenting Wang, Xiaolong Wang, Zhiyong Wang, Wei Wei, Bin Wen, Carole-Jean Wu, Eric Xu, Bi Xue, Zheng Yan, Chao Yang, Jun-

jie Yang, Zimeng Yang, Chunxing Yin, Daniel Yin, Yiling You, Keke Zhai, Yanli Zhao, Zhuoran Zhao, Hui Zhang, Jingjing Zhang, Lu Zhang, Lujia Zhang, Na Zhang, Rui Zhang, Xiong Zhang, Ying Zhang, Zhiyun Zhang, Charles Zheng, Erheng Zhong, Xin Zhuang. 我们谨向 Shikha Kapoor, Rex Cheung, Lana Dam, Ram Ramanathan, Nipun Mathur, Bo Feng, Yanhong Wu, Zhaohui Guo, Hongjie Bai, Wen-Yun Yang, Zellux Wang, Arun Singh, Bruce Deng, Yisong Song, Haotian Wu, Meihong Wang 在产品支持方面的帮助，以及 Joseph Laria, Akshay Hegde, Abha Jain, Raj Ganapathy 在项目管理方面的协助表示感谢。最后，我们感谢 Ajit Mathews, Shilin Ding, Hong Yan, Lars Backstrom 提供的领导支持，以及与 Andrew Tulloch, Liang Xiong, Kaushik Veeraraghavan, Gaofeng Zhao 开展的富有启发性的讨论。

参考文献

- Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In Proceedings of the 17th ACM Conference on Recommender Systems, RecSys '23. ACM, September 2023. doi: 10.1145/3604915.3608857. URL <http://dx.doi.org/10.1145/3604915.3608857>.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. 2020.
- Chang, J., Zhang, C., Fu, Z., Zang, X., Guan, L.,

- Lu, J., Hui, Y., Leng, D., Niu, Y., Song, Y., and Gai, K. Twin: Two-stage interest network for lifelong user behavior modeling in ctr prediction at kuaishou, 2023.
- Chen, Q., Zhao, H., Li, W., Huang, P., and Ou, W. Behavior sequence transformer for e-commerce recommendation in alibaba. In Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data, DLP-KDD '19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367837. doi: 10.1145/3326937.3341261. URL <https://doi.org/10.1145/3326937.3341261>.
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. In Proceedings of the 37th International Conference on Machine Learning, ICMML'20, 2020.
- Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. Wide & deep learning for recommender systems. In Proceedings of the 1st Workshop on Deep Learning for Recommender Systems, DLRS 2016, pp. 7–10, 2016. ISBN 9781450347952.
- Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM Conference on Recommender Systems, RecSys '16, pp. 191–198, 2016. ISBN 9781450340359.
- Cui, Z., Ma, J., Zhou, C., Zhou, J., and Yang, H. M6-rec: Generative pretrained language models are open-ended recommender systems, 2022.
- Dallmann, A., Zoller, D., and Hotho, A. A case study on sampling strategies for evaluating neural sequential item recommendation models. In Proceedings of the 15th ACM Conference on Recommender Systems, RecSys '21, pp. 505–514, 2021. ISBN 9781450384582.
- Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems, 2022.
- Devlin, J., Chang, M., Lee, K., and Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL <https://doi.org/10.18653/v1/n19-1423>.
- Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M., and Leskovec, J. Pixie: A system for recommending 3+ billion items to 200+ million users in real-time. In Proceedings of the 2018 World Wide Web Conference, WWW '18, pp. 1775–1784, 2018. ISBN 9781450356398.
- Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. CoRR, abs/1702.03118, 2017. URL <http://arxiv.org/abs/1702.03118>.
- Gao, W., Fan, X., Wang, C., Sun, J., Jia, K., Xiao, W., Ding, R., Bin, X., Yang, H., and Liu, X.

- Learning an end-to-end structure for retrieval in large-scale recommendations. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management, CIKM '21*, pp. 524–533, 2021. ISBN 9781450384469.
- Gillenwater, J., Kulesza, A., Fox, E., and Taskar, B. Expectation-maximization for learning determinantal point processes. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2, NIPS'14*, pp. 3149–3157, Cambridge, MA, USA, 2014. MIT Press.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=uYLFoz1vIAC>.
- Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: A factorization-machine based neural network for ctr prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence, IJCAI'17*, pp. 1725–1731, 2017. ISBN 9780999241103.
- Gupta, M. R., Bengio, S., and Weston, J. Training highly multiclass classifiers. *J. Mach. Learn. Res.*, 15(1):1461–1492, jan 2014. ISSN 1532-4435.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.
- He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., and Candela, J. Q. Practical lessons from predicting clicks on ads at facebook. In *ADKDD'14: Proceedings of the Eighth International Workshop on Data Mining for Online Advertising*, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450329996.
- Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. Session-based recommendations with recurrent neural networks. In Bengio, Y. and LeCun, Y. (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL <http://arxiv.org/abs/1511.06939>.
- Hua, W., Dai, Z., Liu, H., and Le, Q. V. Transformer quality in linear time. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA, volume 162 of Proceedings of Machine Learning Research*, pp. 9099–9117. PMLR, 2022. URL <https://proceedings.mlr.press/v162/hua22a.html>.
- Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Deep networks with stochastic depth, 2016.
- Jegou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, jan 2011. ISSN 0162-8828. doi: 10.1109/TPAMI.2010.57. URL <https://doi.org/10.1109/TPAMI.2010.57>.
- Kang, W.-C. and McAuley, J. Self-attentive sequential recommendation. In *2018 International Conference on Data Mining (ICDM)*, pp. 197–206, 2018.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.

- Khudia, D., Huang, J., Basu, P., Deng, S., Liu, H., Park, J., and Smelyanskiy, M. Fbgemm: Enabling high-performance low-precision deep learning inference. *arXiv preprint arXiv:2101.05615*, 2021.
- Klenitskiy, A. and Vasilev, A. Turning dross into gold loss: is bert4rec really better than sasrec? In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys '23*, pp. 1120–1125, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702419. doi: 10.1145/3604915.3610644. URL <https://doi.org/10.1145/3604915.3610644>.
- Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022.
- Li, C., Chang, E., Garcia-Molina, H., and Wiederhold, G. Clustering for approximate similarity search in high-dimensional spaces. *IEEE Transactions on Knowledge and Data Engineering*, 14(4):792–808, 2002.
- Li, J., Wang, M., Li, J., Fu, J., Shen, X., Shang, J., and McAuley, J. Text is all you need: Learning language representations for sequential recommendation. In *KDD*, 2023.
- Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., and Cheng, Y. Monolith: Real time recommendation system with collisionless embedding table, 2022.
- Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. *KDD '18*, 2018.
- Mudigere, D., Hao, Y., Huang, J., Jia, Z., Tulloch, A., Sridharan, S., Liu, X., Ozdal, M., Nie, J., Park, J., Luo, L., Yang, J. A., Gao, L., Ivchenko, D., Basant, A., Hu, Y., Yang, J., Ardestani, E. K., Wang, X., Komuravelli, R., Chu, C.-H., Yilmaz, S., Li, H., Qian, J., Feng, Z., Ma, Y., Yang, J., Wen, E., Li, H., Yang, L., Sun, C., Zhao, W., Melts, D., Dhulipala, K., Kishore, K., Graf, T., Eisenman, A., Matam, K. K., Gangidi, A., Chen, G. J., Krishnan, M., Nayak, A., Nair, K., Muthiah, B., khorashadi, M., Bhattacharya, P., Lapukhov, P., Naumov, M., Mathews, A., Qiao, L., Smelyanskiy, M., Jia, B., and Rao, V. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA '22*, pp. 993–1011, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450386104. doi: 10.1145/3470496.3533727. URL <https://doi.org/10.1145/3470496.3533727>.
- Peng, B., Quesnelle, J., Fan, H., and Shippole, E. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=wHBfxhZu1u>.
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022.
- Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.

- Rabe, M. N. and Staats, C. Self-attention does not need $o(n^2)$ memory, 2021.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), jan 2020. ISSN 1532-4435.
- Rendle, S. Factorization machines. In 2010 IEEE International Conference on Data Mining (ICDM), pp. 995–1000, 2010. doi: 10.1109/ICDM.2010.127.
- Rendle, S., Krichene, W., Zhang, L., and Anderson, J. Neural collaborative filtering vs. matrix factorization revisited. In Fourteenth ACM Conference on Recommender Systems (RecSys’20), pp. 240–248, 2020. ISBN 9781450375832.
- Shazeer, N. Glu variants improve transformer, 2020.
- Shin, K., Kwak, H., Kim, S. Y., Ramström, M. N., Jeong, J., Ha, J.-W., and Kim, K.-M. Scaling law for recommendation models: towards general-purpose user representations. In Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’23/IAAI’23/EAAI’23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i4.25582. URL <https://doi.org/10.1609/aaai.v37i4.25582>.
- Shrivastava, A. and Li, P. Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips). In Advances in Neural Information Processing Systems, volume 27, 2014.
- Sileo, D., Vossen, W., and Raymaekers, R. Zero-shot recommendation as language modeling. In Hagen, M., Verberne, S., Macdonald, C., Seifert, C., Balog, K., Nørkvåg, K., and Setty, V. (eds.), Advances in Information Retrieval - 44th European Conference on IR Research, ECIR 2022, Stavanger, Norway, April 10-14, 2022, Proceedings, Part II, volume 13186 of Lecture Notes in Computer Science, pp. 223–230. Springer, 2022. doi: 10.1007/978-3-030-99739-7_26. URL https://doi.org/10.1007/978-3-030-99739-7_26.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023.
- Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM ’19, pp. 1441–1450, 2019. ISBN 9781450369763.
- Tang, H., Liu, J., Zhao, M., and Gong, X. Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations. In Proceedings of the 14th ACM Conference on Recommender Systems, RecSys ’20, pp. 269–278, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450375832. doi: 10.1145/3383313.3412236. URL <https://doi.org/10.1145/3383313.3412236>.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller,

- B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023b.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, pp. 6000–6010, 2017. ISBN 9781510860964.
- Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In Proceedings of the Web Conference 2021, WWW '21, pp. 1785–1797, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3450078. URL <https://doi.org/10.1145/3442381.3450078>.
- Wang, Z., Zhao, L., Jiang, B., Zhou, G., Zhu, X., and Gai, K. Cold: Towards the next generation of pre-ranking system, 2020.
- Xia, X., Eksombatchai, P., Pancha, N., Badani, D. D., Wang, P.-W., Gu, N., Joshi, S. V., Farahpour, N., Zhang, Z., and Zhai, A. Transact: Transformer-based realtime user action model for recommendation at pinterest. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '23, pp. 5249–5259, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701030. doi: 10.1145/3580305.3599918. URL <https://doi.org/10.1145/3580305.3599918>.
- Xiao, J., Ye, H., He, X., Zhang, H., Wu, F., and Chua, T.-S. Attentional factorization machines: Learning the weight of feature interactions via attention networks. In Proceedings of the 26th International Joint Conference on Artificial Intelligence, IJCAI'17, pp. 3119–3125. AAAI Press, 2017. ISBN 9780999241103.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. On layer normalization in the transformer architecture. In Proceedings of the 37th International Conference on Machine Learning, ICML'20. JMLR.org, 2020.
- Yang, J., Yi, X., Zhiyuan Cheng, D., Hong, L., Li, Y., Xiaoming Wang, S., Xu, T., and Chi, E. H. Mixed negative sampling for learning two-tower neural networks in recommendations. In Companion Proceedings of the Web Conference 2020, WWW '20, pp. 441–447, 2020. ISBN 9781450370240.
- Zhai, J., Lou, Y., and Gehrke, J. Atlas: A probabilistic algorithm for high dimensional similarity search. In Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, SIGMOD '11, pp. 997–1008, 2011. ISBN 9781450306614.
- Zhai, J., Gong, Z., Wang, Y., Sun, X., Yan, Z., Li, F., and Liu, X. Revisiting neural retrieval on accelerators. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '23, pp. 5520–5531, New York, NY, USA, 2023a. Association for Computing Machinery. ISBN 9798400701030. doi:

- 10.1145/3580305.3599897. URL <https://doi.org/10.1145/3580305.3599897>.
- Zhai, Y., Jiang, C., Wang, L., Jia, X., Zhang, S., Chen, Z., Liu, X., and Zhu, Y. Byte-transformer: A high-performance transformer boosted for variable-length inputs. In 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 344–355, Los Alamitos, CA, USA, may 2023b. IEEE Computer Society. doi: 10.1109/IPDPS54959.2023.00042. URL <https://doi.ieeecomputersociety.org/10.1109/IPDPS54959.2023.00042>.
- Zhang, B., Luo, L., Liu, X., Li, J., Chen, Z., Zhang, W., Wei, X., Hao, Y., Tsang, M., Wang, W., Liu, Y., Li, H., Badr, Y., Park, J., Yang, J., Mudigere, D., and Wen, E. Dhen: A deep and hierarchical ensemble network for large-scale click-through rate prediction, 2022.
- Zhao, X., Xia, L., Zhang, L., Ding, Z., Yin, D., and Tang, J. Deep reinforcement learning for page-wise recommendations. In Proceedings of the 12th ACM Conference on Recommender Systems, RecSys ’18, pp. 95–103, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450359016. doi: 10.1145/3240323.3240374. URL <https://doi.org/10.1145/3240323.3240374>.
- Zhao, Z., Yang, Y., Wang, W., Liu, C., Shi, Y., Hu, W., Zhang, H., and Yang, S. Breaking the curse of quality saturation with user-centric ranking, 2023.
- Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., and Gai, K. Deep interest network for click-through rate prediction. KDD ’18, 2018.
- Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In Proceedings of the 29th ACM International Conference on Information & Knowledge Management, CIKM ’20, pp. 1893–1902, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368599. doi: 10.1145/3340531.3411954. URL <https://doi.org/10.1145/3340531.3411954>.
- Zhuo, J., Xu, Z., Dai, W., Zhu, H., Li, H., Xu, J., and Gai, K. Learning optimal tree models under beam search. In Proceedings of the 37th International Conference on Machine Learning, ICML’20. JMLR.org, 2020.

A. 符号说明

我们在 Table 8 和 Table 9 中汇总了本文使用的关键符号。

B. 生成式推荐器：背景与形式化

许多读者可能更熟悉经典深度学习推荐模型 (DLRM) (Mudigere et al., 2022), 因为它从 YouTube DNN 时代起就广受欢迎 (Covington et al., 2016), 并被所有大型在线内容和电子商务平台广泛使用 (Cheng et al., 2016; Zhou et al., 2018; Wang et al., 2021; Chang et al., 2023; Xia et al., 2023; Zhai et al., 2023a)。DLRM 在异质特征空间之上运行, 并使用多种神经网络, 包括特征交互模块 (Guo et al., 2017; Xiao et al., 2017; Wang et al., 2021)、序列池化或目标感知的成对注意力模块 (Hidasi et al., 2016; Zhou et al., 2018; Chang et al., 2023), 以及高级多专家多任务模块 (Ma et al., 2018; Tang et al., 2020)。因此, 我们在 Section 2 和 Section 3 中通过与经典 DLRM 明确对比, 概述了生成式推荐器 (GR)。本节从经典序列推荐文献出发, 为读者提供另一种视角。

B.1. 背景：学术界与工业界的序列推荐

B.1.1. 学术研究（传统序列推荐器设定）

循环神经网络 (RNN) 最早由 GRU4Rec (Hidasi et al., 2016) 应用于推荐场景。Hidasi et al. (2016) 研究了门控循环单元 (GRU), 并将其应用于两个数据集: RecSys Challenge 2015² 和 VIDEO (一个专有数据集)。在这两种情况下, 输入序列中都只保留正向事件 (被点击的电商商品, 或用户观看时长至少达到某个阈值的视频)。我们进一步观察到, 在由召回和排序阶段组成的经典工业级两阶段推荐系统设置中 (Covington et al., 2016), Hidasi et al. (2016) 所解决的任务主要对应召回任务。

Transformer、序列转导架构及其变体。此后序列转导架构的发展, 尤其是 Transformer (Vaswani et al., 2017), 推动了推荐系统中的类似进展。SASRec (Kang & McAuley, 2018) 首次是自回归设置中应用 Transformer。它们将是否存在评论或评分视为正向反馈, 从而把 Amazon Reviews³ 和 MovieLens⁴ 等经典数据集转换为正向物品序列, 这与 GRU4Rec 类似。其采用二元交叉熵损失, 其中正目标被定义为下一个“正向”物品 (回顾一下, 这本质上只是存在评论或评分), 负目标则从物品语料库中随机采样 $\mathbb{X} = \mathbb{X}_c$ 。

此后的大多数研究都建立在上述与 GRU4Rec (Hidasi et al., 2016) 和 SASRec (Kang & McAuley, 2018) 类似的设置之上, 例如 BERT4Rec (Sun et al., 2019) 应用了 BERT (Devlin et al., 2019) 的双向编码器设置, S3Rec (Zhou et al., 2020) 引入了显式预训练阶段, 等等。

B.1.2. 作为深度学习推荐模型 (DLRM) 组成部分的工业应用。

包括序列编码器和成对注意力模块在内的序列方法, 因能够作为 DLRM 的组成部分增强用户表示, 已广泛应用于工业场景。DLRM 通常使用相对较短的序列长度, 例如 BST (Chen et al., 2019) 中为 20, DIN (Zhou et al., 2018) 中为 1,000, TransAct (Xia et al., 2023) 中为 100。我们注意到, 与本工作中的 8,192 (Section 4.3) 相比, 这些长度小了 1-3 个数量级。

²<http://2015.recsyschallenge.com/>

³<https://jmcauley.ucsd.edu/data/amazon/>

⁴<https://grouplens.org/datasets/movielens/1m/>, <https://grouplens.org/datasets/movielens/20m/>

尽管使用较短的序列长度，大多数 DLRM 仍能成功捕获用户的长期偏好。这可归因于两个关键方面。第一，现代 DLRM 通常使用预计算的用户画像/嵌入 (Xia et al., 2023) 或外部向量存储 (Chang et al., 2023)，二者都能有效扩展回溯窗口。第二，通常会使用大量上下文侧、用户侧和物品侧特征 (Zhou et al., 2018; Chen et al., 2019; Chang et al., 2023; Xia et al., 2023)，并利用 FM (Xiao et al., 2017; Guo et al., 2017)、DCN (Wang et al., 2021)、MoE 等各种异构网络来变换表示并组合输出。

与 Appendix B.1.1 中讨论的序列设置不同，所有主要工业工作都在 (用户/请求, 候选物品) 对上定义损失。在排序设置中，通常采用多任务二元交叉熵损失。在召回设置中，双塔设置 (Covington et al., 2016) 仍占主导地位。近期工作探索了将下一个待推荐物品表示为 (子) token 序列上的概率分布，例如 OTM (Zhuo et al., 2020) 和 DR (Gao et al., 2021) (请注意，在其他近期工作中，同一设置有时也称为“生成式召回”)。它们通常使用束搜索从子 token 中解码物品。随着 GPU、定制 ASIC 和 TPU 等现代加速器日益普及，混合 logits (Zhai et al., 2023a) 等高级学习型相似度函数也已被提出并部署，作为双塔设置和束搜索的替代方案。

从问题建模的角度看，我们将上述所有工作都视为 DLRM (Mudigere et al., 2022) 的一部分，因为所用模型架构、特征和损失与 Appendix B.1.1 中讨论的学术界序列推荐研究存在显著差异。还值得指出的是，在本工作之前，业界尚未成功应用完全序列化的排序设置，尤其未曾达到十亿日活用户 (DAU) 的规模。

B.2. 形式化：将生成式推荐器 (GR) 中的排序与检索建模为序列转导任务

下面讨论传统序列推荐器设定和 DLRM 设定中的三项局限，以及生成式推荐器 (GR) 如何从问题表述角度解决这些局限。

忽略用户交互物品之外的特征。以往的序列表述只考虑用户明确交互过的内容 (物品) (Hidasi et al., 2016; Kang & McAuley, 2018; Sun et al., 2019; Zhou et al., 2020)、而在 GR 出现之前，工业规模的推荐系统会基于数量庞大的特征进行训练，以增强用户和内容的表示 (Covington et al., 2016; Cheng et al., 2016; Zhou et al., 2018; Chen et al., 2019; Chang et al., 2023; Xia et al., 2023; Zhai et al., 2023a)。GR 通过以下方式解决这一局限：a) 压缩其他类别特征并将其与主时间序列合并；b) 如 Section 2.1 和 Figure 2 所述，采用目标感知表述，通过交叉注意力交互捕获数值特征。我们的验证表明，忽略这些特征的传统“仅交互”表述会显著降低模型质量；实验结果见 Table 7 和 Table 6 中标为“GR (仅交互)”的行。结果显示，仅使用交互历史会使检索的命中率 @100 下降 1.3%，并使排序的 NE 下降 2.6% (回顾 Sections 4.1.2 and 4.3.1 的讨论，NE 变化 0.1% 即具有显著意义)。

用户表示在与目标无关的设定下计算。第二个问题是，大多数传统序列推荐器，包括 GRU4Rec (Hidasi et al., 2016)、SASRec (Kang & McAuley, 2018)、BERT4Rec (Sun et al., 2019)、S3Rec (Zhou et al., 2020)、等，都采用与目标无关的表述：对于目标物品 Φ_i ，使用 $\Phi_0, \Phi_1, \dots, \Phi_{i-1}$ 作为编码器输入来计算用户表示，随后用该表示进行预测。相比之下，工业环境中使用的大多数主流 DLRM 方法都以目标感知方式表述其序列模块，能够将“目标” (排序候选) 信息纳入用户表示。其中包括 DIN (Zhou et al., 2018) (Alibaba)、BST (Chen et al., 2019) (Alibaba)、TWIN (Chang et al., 2023) (Kwai) 和 TransAct (Xia et al., 2023) (Pinterest)。

生成式推荐器 (GR) 通过交错排列内容序列与行为序列 (Section 2.2)，兼具两类方法的优势，从而能在

因果自回归设定中应用目标感知注意力。我们在 Table 10 中对以往工作与本文工作进行了分类和对比⁵。

判别式表述将既有序列推荐器工作的适用范围限制在逐点式设定。最后，传统序列推荐器在设计上属于判别式模型。现有序列推荐器文献，包括 GRU4Rec 和 SASRec 等奠基性工作，对 $p(\Phi_i|\Phi_0, a_0, \dots, \Phi_{i-1}, a_{i-1})$ 建模，即在给定用户当前状态的条件下，对下一个待推荐物品的条件分布进行建模。另一方面，我们观察到，标准推荐系统中存在两个概率过程，即推荐系统向用户推荐内容 Φ_i （例如某张照片或某段视频）的过程，以及用户对所推荐内容 Φ_i 通过某种行为 a_i 作出反应的过程（该行为可以是点赞、完整观看视频、跳过等行为的组合）。

生成式方法需要对所推荐内容与用户行为的联合序列分布，即 $p(\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-1}, a_{n_c-1})$ ，进行建模，详见 Section 2.2。我们提出的生成式推荐器能够对此类分布进行建模，如 Table 11 (Figure 8) 所示。下一行为 token (a_i) 预测任务恰好对应 Table 1 中讨论的 GR 排序设定；而下一内容 (Φ_i) 预测任务则类似于适配到交错设定的检索任务，其目标经调整后用于学习输入数据分布。

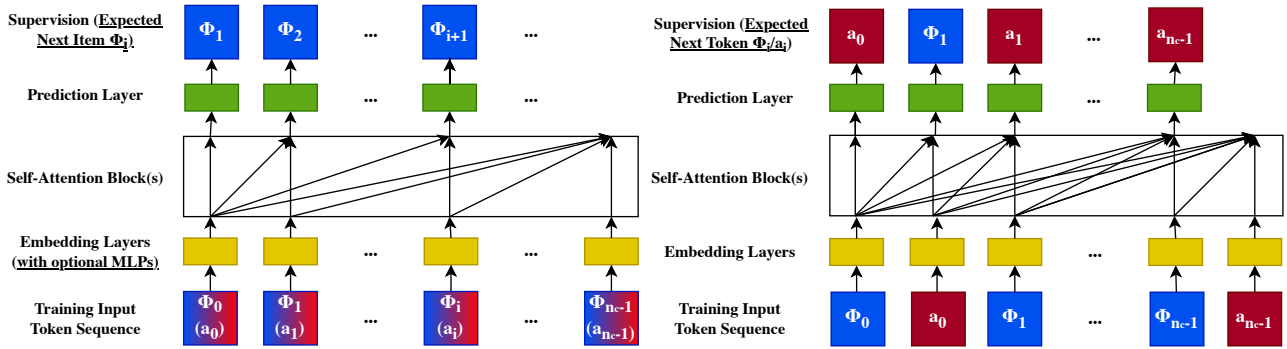


Figure 8. 传统序列推荐器（左）与生成式推荐器（右）的比较。为便于比较，我们展示了因果自回归设置下的序列推荐器，以及不含上下文特征的 GR。在左侧，动作类型 a_i s 要么被忽略，要么先通过 MLP 与物品信息 Φ_i s 结合，再送入自注意力模块。

重要的是，这一表述不仅能恰当地建模数据分布，还能通过波束搜索等方法直接采样待推荐给用户的物品序列。我们推测，相较于传统的列表式设定（例如 DPP (Gillenwater et al., 2014) 和 RL (Zhao et al., 2018)），这将带来更优的方法；对此类系统的完整表述与评估（在 Section 6 中有简要讨论）留作未来工作。

C. 评估：合成数据

如 Section 3.1 所述，标准 softmax 注意力由于其归一化因子的存在，难以捕获用户偏好的强度，而这对用户表示学习非常重要。这一点在推荐场景中尤为关键，因为系统除了需要预测物品的相对顺序，还可能需预测交互强度（例如未来针对某一特定主题的正向动作数量）。

为理解这一行为，我们按照 Dirichlet 过程构造合成数据，该过程在动态词表集合上生成流式数据。Dirichlet 过程能够刻画用户交互历史中“富者愈富”的行为。我们按如下方式设置合成实验：

- 我们将 20,000 个物品 id 中的每一个随机且唯一地分配到 100 个类别之一。

⁵由于日志数据量极其庞大，大多数大规模工业推荐器都需要在流式/单遍设定下训练。

⁶BERT4Rec 采用多遍训练，并混合使用 Cloze 监督损失与逐点式（最后一个物品）监督损失；S3Rec 采用多遍训练，将预训练和微调作为两个独立阶段。

- 我们生成 1,000,000 条长度均为 128 的记录，前 90% 用于训练，最后 10% 用于测试。为模拟流式训练设置，我们初始仅开放 40% 的物品 id，其余 id 以相等间隔逐步开放；即在第 500,000 条记录处，可采样的最大 id 为 $(40\% + 60\% * 0.5) * 20,000 = 14,000$ 。
- 对每条记录，我们从 100 个类别中随机选择至多 5 个，并在这 5 个类别上随机采样先验 H_c 。随后按照如下定义在候选类别上的 Dirichlet 过程，为每个位置依次采样类别：
 - 对于 $n > 1$:
 - * 以概率 $\alpha/(\alpha + n - 1)$ 抽取类别 c ，该类别来自 H_c 。
 - * 以概率 $n_c/(\alpha + n - 1)$ 抽取类别 c ，其中 n_c 是此前类别为 c 的物品数量。
 - * 在满足流式约束的前提下，随机采样一个与类别 c 匹配的已分配物品。

其中， α 从 (1.0, 500.0) 中均匀随机采样。

结果见表 2。由于该数据集不含时间戳，我们始终对 HSTU 消融 $\text{rab}^{p,t}$ 。我们观察到，相较标准 Transformer，HSTU 将 Hit Rate@10 提升了 100% 以上。重要的是，将 HSTU 的逐点注意力机制替换为 softmax (“HSTU w/ Softmax”) 也会显著降低命中率，验证了类逐点注意力聚合机制的重要性。

D. 评估：传统序列推荐器设置

我们在 Section 4.1.1 中的评估重点将 HSTU 与采用最新训练方案的先进 Transformer 基线 SASRec 进行比较。本节进一步考察另外两种替代方法。

循环神经网络 (RNN)。我们考察经典序列推荐工作 GRU4Rec (Hidasi et al., 2016)，以帮助读者理解：当充分纳入所有最新建模和训练改进时，包括 Transformer 和 HSTU 在内的自注意力模型与传统 RNN 相比表现如何。

自监督序列方法。我们考察最流行的工作 BERT4Rec (Sun et al., 2019)，以理解双向自监督 (BERT4Rec 通过 Cloze 目标利用该机制) 与 SASRec 和 HSTU 这类单向因果自回归设置的比较。

结果见 Table 12。我们复用 Klenitskiy & Vasilev (2023) 报告的 BERT4Rec 结果，以及 ML-1M 和 ML-20M 上的 GRU4Rec 结果。由于使用了采样 softmax 损失，我们将负样本数保持不变 (ML-1M、ML-20M 为 128，Amazon Books 为 512)，以确保各方法之间的公平比较。

结果证实，在使用采样 softmax 损失时 (Zhai et al., 2023a; Klenitskiy & Vasilev, 2023)，SASRec 仍是传统序列推荐设置中最具竞争力的方法之一，而 HSTU 显著优于所评估的 Transformer、RNN 和自监督双向 Transformer。

E. 评估：传统 DLRM 基线

Section 4 中使用的 DLRM 基线配置，反映了数百名研究人员和工程师多年来持续迭代的成果，并且高度近似于 HSTU/GR 部署之前、一个拥有数十亿日活用户的大型互联网平台上的生产配置。下面我们对所用模型进行概要说明。

排序设置。如 (Mudigere et al., 2022) 所述，基线排序模型采用约一千个稠密特征和五十个稀疏特征。我们纳入了多种建模技术，例如专家混合 (Ma et al., 2018)、Deep & Cross Network 的各种变体 (Wang

et al., 2021)、包括目标感知成对注意力在内的各种序列推荐模块（工业场景中常用的一种变体见 (Zhou et al., 2018)），以及特殊交互层上的残差连接 (He et al., 2015; Zhang et al., 2022)。对于缩放定律一节 (Section 4.3.1) 中的低 FLOPs 区域，我们简化了一些计算成本较高的模块，和/或将其替换为 DCN 等其他最先进的变体，以达到所需的 FLOPs。

出于保密考虑，我们无法披露确切设置；但据我们所知，当充分纳入近期研究成果后，我们的基线代表了当前已知最佳的 DLRM 方法之一。为验证这一说法并帮助读者理解，我们报告了一种基于相同特征、但仅采用包括 DIN (Zhou et al., 2018)、DCN (Wang et al., 2021) 和 MMoE (Ma et al., 2018) 在内的主要已发表成果的典型设置 (“DLRM (DIN+DCN)”)，结果见 Table 7，其组合架构如 Figure 9 所示。该设置显著逊于我们的生产 DLRM 设置：主 E-Task 的 NE 低 0.71%，主 C-Task 的 NE 低 0.57%（其中 0.1% 的 NE 差异即具有显著意义）。

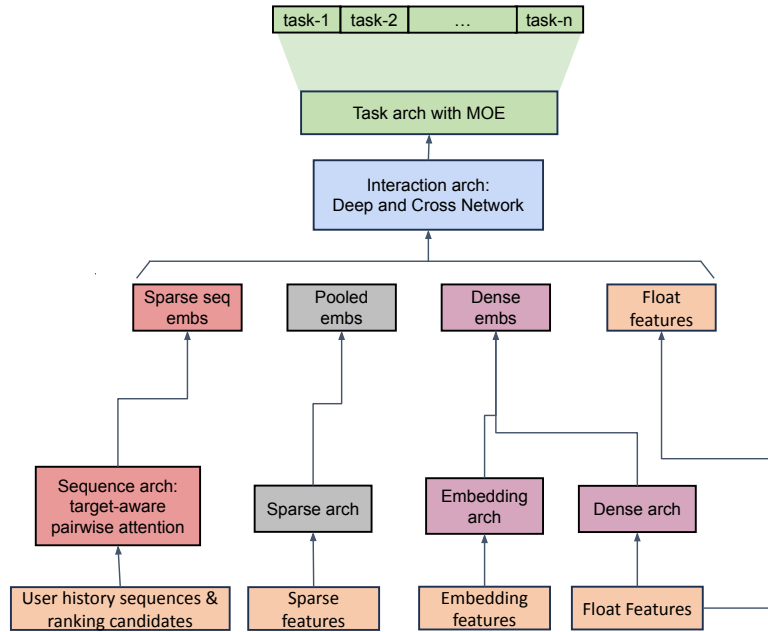


Figure 9. 基线 DLRM 排序模型 (Table 7 中的 “DLRM (DIN+DCN)”) 的高层架构，该模型采用了 DIN (Zhou et al., 2018)、DCN (Wang et al., 2021) 和 MMoE (Ma et al., 2018) 等主要已发表工作。

检索设定。基线检索模型采用标准双塔神经检索设定 (Covington et al., 2016)，并混合使用批内采样和批外采样。输入特征集既包含高基数稀疏特征（例如物品 ID、用户 ID），也包含低基数稀疏特征（例如语言、主题、兴趣实体）。使用一组带残差连接的前馈层 (He et al., 2015)，将输入特征压缩为用户嵌入和物品嵌入。

特征与序列长度。两种 DLRM 基线使用的特征（包括由各类序列编码器/成对注意力模块使用的主要用户交互历史）均是所有 GR 候选模型所用特征的严格超集。这适用于本文开展的所有研究，包括扩展性研究 (Section 4.3.1)。

F. 随机长度

F.1. 子序列选择

在 Equation (4) 中，我们从完整用户历史中选择长度为 L 的子序列，以提高稀疏性。经验结果表明，精心设计子序列选择技术可以提升模型质量。我们计算指标 $f_i = t_n - t_i$ ，它对应于用户与物品 x_i 交互后所经过的时间。我们使用以下子序列选择方法开展离线实验：

- **贪心选择** – 选择 L 个 f_i 值最小的物品（来自 S ）
- **随机选择** – 随机选择 L 个物品（来自 S ）
- **特征加权选择** – 选择 L 个物品（来自 S ，依据加权分布 $1 - f_{n,i}/(\sum_{j=1}^L f_{j,i})$ ）

在离线实验中，特征加权子序列选择方法取得了最佳模型质量，如 Table 13 所示。

F.2. 随机长度对序列稀疏性的影响

在 Table 3 中，我们展示了随机长度对一个具有 30 天用户交互历史的代表性工业规模配置的序列稀疏度影响。序列稀疏度定义为 1 减去所有样本的平均序列长度与最大序列长度之比。为了更好地描述稀疏注意力的计算成本，我们还定义了 s2，即 1 减去注意力矩阵的稀疏度。作为参考，我们分别在 Table 14 和 Table 15 中给出了 60 天和 90 天用户交互历史的结果。

F.3. 与序列长度外推技术的比较

我们开展额外研究，以验证随机长度相较语言建模中现有序列长度外推技术的竞争力。许多现有方法通过修改 RoPE (Su et al., 2023) 实现序列长度外推。为作比较，我们训练了一个不含相对注意力偏置、采用旋转嵌入的 HSTU 变体 (HSTU-RoPE)。

我们在 HSTU-RoPE 上评估以下序列长度外推方法：

- **零样本** - 应用 NTK-Aware RoPE (Peng et al., 2024) 后，不经微调直接评估模型；
- **微调** - 应用 NTK-by-parts (Peng et al., 2024) 后，对模型微调 1000 步。

我们在 HSTU（包含相对注意力偏置，不含旋转嵌入）上评估以下方法：

- **零样本** - 根据最大训练序列长度截断相对位置偏置，并直接评估模型 (Raffel et al., 2020; Press et al., 2022)；
- **微调** - 根据最大训练序列长度截断相对位置偏置，在评估前对模型微调 1000 步。

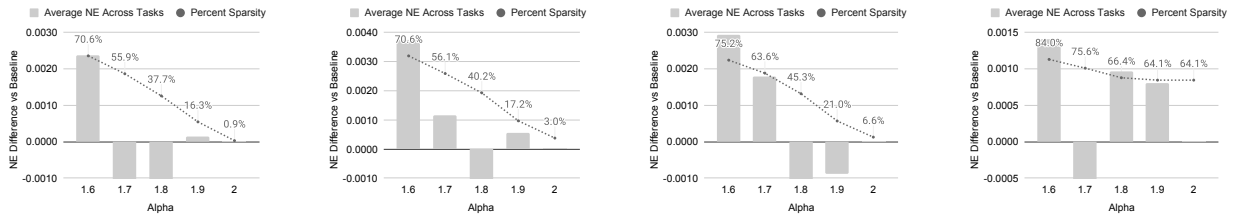


Figure 10. 随机长度 (SL) 对排序模型指标的影响。从左至右： $n = [1024, 2048, 4096, 8192]$ （如 Section 2.2 所述， n 为经交错算法处理后的结果，从而在采用因果掩码的设定中实现目标感知交叉注意力）。

在 Table 16 中，我们报告训练期间引入数据稀疏性的模型（随机长度、零样本、微调）与在完整数据上训练的模型之间的 NE 差异。对于零样本和微调技术，我们将稀疏度定义为训练时平均序列长度除以评估时最大序列长度。所有零样本和微调模型均在序列长度 1024 的数据上训练，并在长度 2048 和 4096 的数据上评估。为找到合适的随机长度基线，我们选择产生相同数据稀疏度指标的随机长度设定。

我们认为零样本和微调式长度外推不适合处理高基数 ID 的推荐场景。经验上，随机长度显著优于这两种方法。这可能源于词表规模庞大：零样本和微调方法无法为较旧 ID 学到良好表示，因而难以充分利用较长序列中的信息。

G. 稀疏分组 GEMM 与融合相对注意力偏置

我们进一步介绍在 Section 3.2 中提出的高效 HSTU 注意力核。我们的方法建立在 Memory-efficient Attention (Rabe & Staats, 2021) 和 FlashAttention (Dao et al., 2022) 之上，是一种内存高效的自注意力机制：它将输入划分为多个块，并避免在反向传播中实例化大型 $h \times N \times N$ 中间注意力张量。通过利用输入序列的稀疏性，我们可以把注意力计算重新表述为一组形状各异、首尾相接的 GEMM。我们实现了高效 GPU 内核来加速该计算。由于内存访问，相对注意力偏置的构造也是一个瓶颈。为解决这一问题，我们将相对偏置构造与分组 GEMM 融合到单个 GPU 内核中，并在反向传播时利用 GPU 的高速共享内存累积梯度。尽管我们的算法需要在反向传播中重新计算注意力和相对偏置，但它比 Transformer 中使用的标准方法快得多，且占用更少内存。

H. 利用可缓存操作的微批次快速注意力 (M-FALCON)

本节详细描述 Section 3.4 中讨论的 M-FALCON 算法。M-FALCON 的伪代码见 Algorithm 1。M-FALCON 引入了三个关键思想。

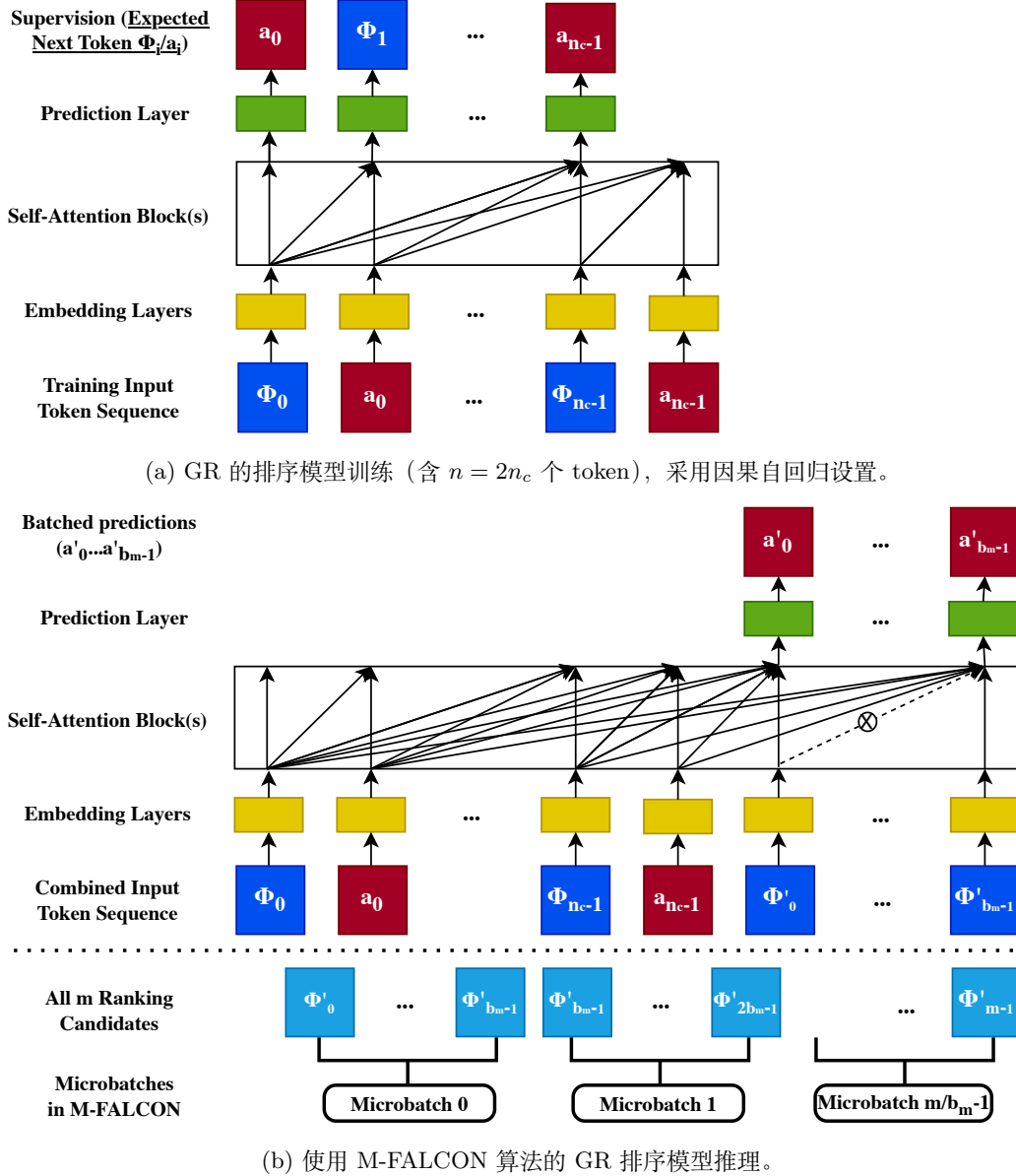


Figure 11. M-FALCON 算法示意图。上：GR 目标感知建模下的模型训练。下：针对 m 个候选 $\Phi'_0, \dots, \Phi'_{m-1}$ 的模型推理，这些候选被划分为 $\lceil m/b_m \rceil$ 个微批次；虚线上方展示了第一个微批次 $\Phi'_0, \dots, \Phi'_{b_m-1}$ 的模型推理（共有 $2n_c + b_m$ 个 token，其中计入了 $\Phi_0, a_0, \dots, \Phi_{n_c-1}, a_{n_c-1}$ ）。请注意，自注意力算法经过修改，使得 Φ'_i 无法关注 Φ'_j 当 $i \neq j$ 时；图中以“ $\times$ ”突出显示了这一点。

批量推理可应用于因果自回归设置。 GR 中的排序任务按 Section 2.2 所述以目标感知方式建模。通常认为，在目标感知设置中，我们需要每次对一个物品进行推理；成本为 $O(mn^2d)$ ，对应 m 个候选项和长度为 n 的序列。我们在此表明这并非最优方案；即使使用原始 Transformer，也可修改自注意力中的注意力掩码，以批量执行这些操作（“批量推理”），并将成本降至 $O((n+m)^2d) = O(n^2d)$ 。

Figure 11 给出了示意图。其中，Figure 11 (a) 和 (b) 都涉及用于因果自回归设置的注意力掩码矩阵。关键区别在于，Figure 11 (a) 在因果训练中使用大小为 $2n_c$ 的标准下三角矩阵，而 Figure 11 (b) 则修改大小为 $2n_c + b_m$ 的下三角矩阵，将 (i, j) 中满足 $i, j \geq 2n_c, i \neq j$ 的位置设为 False 或 $-\infty$ ，以防止目标位置 $\Phi'_0, \dots, \Phi'_{b_m-1}$ 彼此关注。容易看出，通过这种方式，自注意力块对 Φ'_i, a'_i 的输出仅依赖于 $\Phi_0, a_0, \dots, \Phi_{n_c-1}, a_{n_c-1}$ ，而不依赖于 Φ'_j ($i \neq j$)。换言之，通过使用修改后的注意力掩码对 $(2n_c + b_m)$ 个 token 进行一次前向传播，我们现在可以为最后 b_m 个 token 获得相同结果，就如同执行了 b_m 次分别针对 $(2n_c + 1)$ 个 token 的前向传播：将 Φ'_i 放在第 $2n_c$ 个（从 0 开始）位置，并在第 i 次前向传播中使用标准因果注意力掩码。

微批处理将批量推理扩展到大规模候选集。 排序阶段可能需要处理多达数万个排序候选项 (Wang et al., 2020)。我们可将全部 m 个候选项分成 $\lceil m/b_m \rceil$ 个大小为 b_m 的微批，使得 $O(b_m) = O(n)$ ；对于大多数实际推荐设置（候选项可多达数万个），这仍保留了前述 $O((n+m)^2d) = O(n^2d)$ 运行时间。

编码器级缓存使请求内部乃至请求之间均可共享计算。 最后，KV 缓存 (Pope et al., 2022) 既可应用于请求内部，也可应用于请求之间。例如，对于本文提出的 HSTU 模型 (Section 3)， $K(X)$ 和 $V(X)$ 可完全在请求内部和/或请求之间的微批之间缓存。对于缓存的前向传播，我们只需计算 $U(X)$ 、 $Q(X)$ 、 $K(X)$ 和 $V(X)$ ，它们对应最后 b_m 个 token；同时复用已缓存的 $K(X)$ 和 $V(X)$ ，用于包含 n 个 token 的序列化用户历史。 $f_2(\text{Norm}(A(X)V(X)) \odot U(X))$ 同样只需为 b_m 个候选项重新计算。这将缓存前向传播的计算复杂度降至 $O(b_md^2 + b_mnd)$ ；相较 $O((n+b_m)d^2 + (n+b_m)^2d)$ 可显著改善 2-4 倍，即使在 $b_m = n$ 时也是如此。

Algorithm 1 M-FALCON 算法。

- 1: **输入:** 合并后的 token 序列 $x_0, x_1, \dots, x_{n-1}$ (例如可以是 $(\Phi_0, a_0, \dots, \Phi_{n_c-1}, a_{n_c-1})$ ，其中 $n = 2n_c$)； m 个排序候选 $\Phi'_0, \dots, \Phi'_{m-1}$ ；一个在因果自回归设定下训练、具有 b 层和 h 个头的自注意力模型 (例如 HSTU 或 Transformer) $f(X, \text{cacheStates}, \text{attnMask}) \rightarrow (X', \text{updatedCacheStates})$ ，其中 $X, X' \in \mathbb{R}^{N \times d}$ ， $\text{attnMask} \in \mathbb{R}^{N \times N}$ 和 $\text{cachedStates}, \text{updatedCacheStates} \in \mathbb{R}^{b \times h \times N \times d_{qk}} \times \mathbb{R}^{b \times h \times N \times d_{qk}}$ (因为缓存了 $K(X)$ 和 $V(X)$ ，并跨越 b 层)；微批大小 b_m ，为简单起见，我们假设 m 是 b_m 的倍数。
 - 2: **输出:** 所有 m 个排序候选的预测结果 $(a'_0, \dots, a'_{m-1})$ 。
 - 3: $\text{numMicrobatches} = (m + b_m - 1) // b_m$
 - 4: $\text{attnMask} = L_{n+b_m} \{L_{n+b_m} \text{ 表示下三角矩阵。下三角元素为 0，其余元素为 } -\infty.\}$
 - 5: $\text{attnMask}[i, j] = -\infty$ for $i, j \geq n, i \neq j$ {这可以防止最后 b_m 个元素彼此关注。}
 - 6: $(a'_0, a'_1, \dots, a'_{b_m-1}), kvCache \leftarrow f(\text{embLayer}((x_0, x_1, \dots, x_{n-1}, \Phi'_0, \dots, \Phi'_{b_m-1})), \emptyset, \text{attnMask})$
 - 7: $\text{predictions} = (a'_0, a'_1, \dots, a'_{b_m-1})$
 - 8: $i = 1$
 - 9: while $i < \text{numMicrobatches}$ do
 - 10: $(a'_{b_mi}, a'_{b_mi+1}, a'_{b_m(i+1)-1}), _ \leftarrow f(\text{embLayer}((x_0, x_1, \dots, x_{n-1}, \Phi'_{b_mi}, \dots, \Phi'_{b_m(i+1)-1})), kvCache, \text{attnMask})$
 - 11: $\text{predictions} \leftarrow \text{predictions} + (a'_{b_mi}, a'_{b_mi+1}, \dots, a'_{b_m(i+1)-1})$
 - 12: $i \leftarrow i + 1$
 - 13: end while
 - 14: return predictions
-

为帮助理解，Figure 11 对 Algorithm 1 进行了图示。我们指出，M-FALCON 不仅适用于 HSTU 和 GR，还可广泛用作其他基于自注意力架构的目标感知因果自回归设定中的推理优化算法。

H.1. 推理吞吐量评估：采用 M-FALCON 的生成式推荐器（GR）与 DLRM 的比较

如 Section 3.4 所述，M-FALCON 并行处理 b_m 个候选项，以在推理时将计算成本均摊到所有 m 个候选项。为理解我们的设计，我们在相同硬件配置下比较 GR 和 DLRM 的吞吐量（即每秒评分的候选项数，QPS）。

如 Figure 12 和 Figure 13 所示，由于批量推理能够均摊成本，GR 的吞吐量随排序阶段候选项数（ m ）以次线性方式扩展，直至某一区域——在我们的案例研究中为 $m = 2048$ 。这证实了批量推理在因果自回归设置中的关键性。由于注意力复杂度按 $O((n + b_m)^2)$ 扩展，仅使用多个微批就能提高吞吐量。在微批处理之上，缓存进一步消除了冗余的线性计算和注意力计算。如 Figure 13 所示，两者结合后，相对使用单个微批的 $b_m = m = 1024$ 基线，可额外加速最高 1.99x。总体而言，凭借高效的 HSTU 编码器设计并利用 M-FALCON，基于 HSTU 的生成式推荐器在大规模生产配置上的吞吐量最高超过 DLRM 2.99x，尽管 GR 的 FLOPs 复杂度高出 285x。

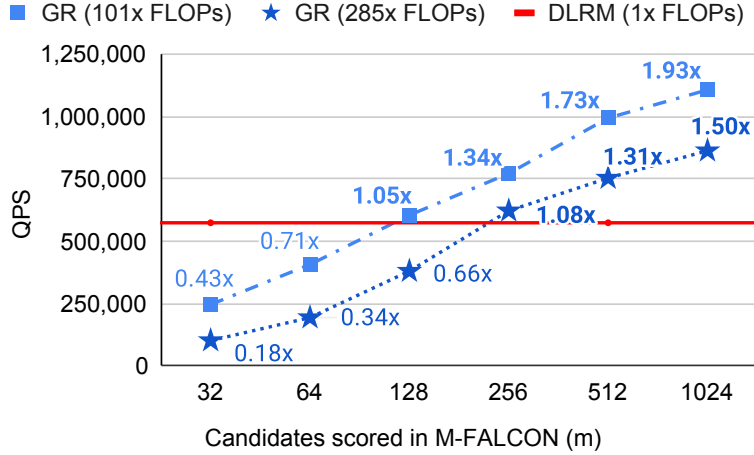


Figure 12. 端到端推理吞吐量：大规模工业设置下的 DLRM 与 GR（使用 M-FALCON）。请注意，该图与 Figure 6 相同，此处复现以方便阅读。

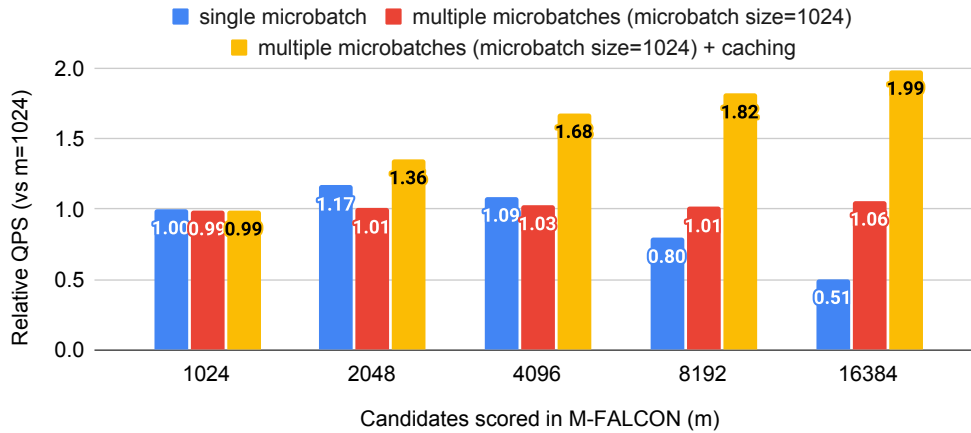


Figure 13. 端到端推理吞吐量：在 285x FLOPs 的 GR 模型上，M-FALCON 在大批量设置中的吞吐量伸缩情况，其中 m （排序候选项总数）范围为 1024 至 16384，且 $b_m = 1024$ 。

符号	描述
$\Psi_k(t_j)$	第 k 个训练样本 (k 按全局排序) 在时间 t_j 由特征日志系统产生。在典型 DLRM 推荐系统中, 用户消费某项内容 Φ_i 后 (以跳过、完整观看视频和分享等行为 a_i 响应), 特征日志系统将元组 (Φ_i, a_i) 与用于对 Φ_i 排序的特征联结, 并将 $(\Phi_i, a_i, \text{features for } \Phi_i)$ 作为训练样本 $\Psi_k(t_j)$ 输出。如 Section 2.3 所述, DLRM 与 GR 处理的训练样本数量不同, GR 的样本通常少 1-2 个数量级。
n_c ($n_{c,i}$)	用户已交互的内容数量 (用户/样本 i)。
$\Phi_0, \dots, \Phi_{n_c-1}$	推荐系统语境下, 用户已交互内容的列表。 与 Φ_i 对应的用户行为列表。当所有预测事件均为二元事件时, 每个行为可视为由点赞、分享、评论、查看图片、开始播放视频、完整观看视频、隐藏等 (原子) 事件构成的多热向量。
$a_0, \dots, a_{n_c-1}$	
E, F	Figure 2 中 DLRM 的类别特征。 $E_0, E_1, \dots, E_7, E_8, F_0, F_1, \dots, F_7$ 表示在不同时间点通过特征提取由 $(\Phi_0, a_0, t_0), \dots, (\Phi_{n_c-1}, a_{n_c-1}, t_{n_c-1})$ 得到的变换 (例如最近点赞的 10 张图片、用户过去点击过且与当前候选最相似的 50 个 URL 等)。“合并 & 序列化”表示获得原始交互序列 $(\Phi_0, a_0, t_0), \dots, (\Phi_{n_c-1}, a_{n_c-1}, t_{n_c-1})$ 的 (虚拟) 逆过程。
G, H	Figure 2 中与用户-内容交互无关的 DLRM 类别特征。如 Section 2.1 所述及 Figure 2 所示, 这些特征 (例如人口统计特征或已关注的创作者) 会被合并到主时间序列 (用户交互过的内容列表, 例如 $\Phi_0, a_0, \dots, \Phi_{n_c-1}, a_{n_c-1}$) 中。
n (n_i)	序列转导任务中的 token 数量 (用户/样本 i)。即使不存在任何与非交互相关的类别特征, $O(n) = O(n_c)$ 、 n 也可能不同于 n_c ; 例如参见 Table 1。
$x_0, \dots, x_{n-1}$	序列转导任务中的输入 token 列表。
$y_0, \dots, y_{n-1}$	序列转导任务中的输出 token 列表。
$t_0, \dots, t_{n-1}$	与观测到 $x_0, \dots, x_{n-1}$ 的时间对应的的时间戳列表。
$\mathbb{X}, \mathbb{X}_c$	所有输入/输出 token 的词表 ($\mathbb{X}$) 及其内容子集 ($\mathbb{X}_c$)。
N, N_c	$\max_i n_i, \max_i n_{c,i}$ 。
u_t	时间 t 的用户表示。
$s_u(n_i), \hat{s}_u(n_i)$	用户 i 的采样率, 用于生成式训练 (Section 2.3)。
d	模型维度 (嵌入维度)。
d_{qk}	HSTU 和 Transformer 中的注意力维度大小。这适用于 Equation (1) 中的 $Q(X)$ 和 $K(X)$ 。
d_v	HSTU 中的值维度大小。对于 Transformer, 通常有 $d_{qk} = d_v$ 。
d_{ff}	Transformer 逐点前馈层中的隐藏维度大小。HSTU 不使用前馈层; 见下文 $U(X)$ 。
h	注意力头数量。
l	HSTU 的层数。对于 Transformer, 注意力层与逐点前馈层共同构成一层。

Table 8. 符号表 (下页续)。

符号	说明
X	HSTU 层的输入。按照标准术语（批处理前）， $X \in \mathbb{R}^{N \times d}$ ，假设我们有一个包含 N 个 token 的输入序列。
$Q(X), K(X), V(X)$	根据 Equation (1)，由给定输入 X 在 HSTU 中得到的查询、键和值。该定义类似于标准 Transformer 中的 Q 、 K 和 V 。 $Q(X), K(X) \in \mathbb{R}^{h \times N \times d_{qk}}$ ，以及 $V(X) \in \mathbb{R}^{h \times N \times d_v}$ 。
$U(X)$	HSTU 在 Equation (3) 中使用 $U(X)$ 对注意力池化值 ($V(X)$) 进行“门控”，它与 $f_2(\cdot)$ 一起使 HSTU 能够完全避免前馈层。 $U(X) \in \mathbb{R}^{h \times N \times d_v}$ 。
$A(X)$	由输入 X 得到的注意力张量。 $A(X) \in \mathbb{R}^{h \times N \times N}$ 。
$Y(X)$	由输入 X 得到的 HSTU 层输出。 $Y(X) \in \mathbb{R}^d$ 。
$\text{Split}(\cdot)$	将张量拆分成多个块的操作。Equation (1) 中的 $\phi_1(f_1(X)) \in \mathbb{R}^{N \times (2hd_{qk} + 2hd_v)}$ ；我们得到 $U(X)$ 、 $V(X)$ （两者形状均为 $h \times N \times d_v$ ）以及 $Q(X)$ 、 $K(X)$ （两者形状均为 $h \times N \times d_{qk}$ ），方法是使用 $U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X)))$ 拆分较大张量（并置换维度）。
$\text{rab}^{p,t}$	同时纳入位置 (Raffel et al., 2020) 和时间信息的相对注意力偏置（基于 token 被观测到的时间， $t_0, \dots, t_{n-1}$ ；一种可行的实现是对 $(t_j - t_i)$ 应用某种分桶函数以得到 (i, j) ）。实际中，我们在一层内的不同注意力头之间共享 $\text{rab}^{p,t}$ ，因此 $\text{rab}^{p,t} \in \mathbb{R}^{1 \times N \times N}$ 。
α	控制 HSTU 中所用 随机长度算法稀疏度的参数 (Section 3.2)。
R	寄存器大小 在 GPU 上，针对 Section 3.2 中讨论的 HSTU 算法。
m	推荐系统排序阶段所考虑的候选项数量。
b_m	M-FALCON 算法见 Section 3.4 的讨论中的微批大小。

Table 9. 符号表（续）

	目标物品 i 的输入	目标物品 i 的期望输出	架构	训练流程
GR	$\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_i$	a_i （目标感知）	自注意力（HSTU）	因果自回归（流式/单遍）
GRU4Rec SASRec	$\Phi_0, \Phi_1, \dots, \Phi_{i-1}$	Φ_i	RNN（GRU） 自注意力（Transformer）	因果自回归（多遍）
BERT4Rec S3Rec	$\Phi_0, \Phi_1, \dots, \Phi_{i-1}$ （推理时）	Φ_i	自注意力（Transformer）	序列式多遍 ⁶
DIN BST TWIN TransAct	$\Phi_0, \Phi_1, \dots, \Phi_i$ $(\Phi_0, a_0), \dots, (\Phi_{i-1}, a_{i-1}), \Phi_i$	a_i （目标感知， 隐式作为 DLRM 的一部分）	成对注意力 自注意力（Transformer） 两阶段成对注意力 自注意力（Transformer）	逐点式（通常为流 式/单遍）

Table 10. 在排序设定下对既有序列推荐器工作与 GR 的比较；为完整起见，亦纳入 DLRM。

任务	规格 (输入 / 输出 / 长度)		
下一行为 token (a_i) 预测	x_i	$\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-2}, a_{n_c-2}, \Phi_{n_c-1}, a_{n_c-1}$	
	y_i	$a_0, \emptyset, a_1, \emptyset, \dots, a_{n_c-2}, \emptyset, a_{n_c-1}, \emptyset$	
	n	$2n_c$	
下一内容 token (Φ_i) 预测	x_i	$\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-2}, a_{n_c-2}, \Phi_{n_c-1}, a_{n_c-1}$	
	y_i	$\emptyset, \Phi_1, \emptyset, \Phi_2, \dots, \emptyset, \Phi_{n_c-1}, \emptyset, \emptyset$	
	n	$2n_c$	

Table 11. 对 $p(\Phi_0, a_0, \dots, \Phi_{n_c-1}, a_{n_c-1})$ 进行生成式建模。示意图见 Figure 8。

	方法	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2853	.5474	.7528	.1603	.2498
	BERT4Rec	.2843 (-0.4%)	—	—	.1537 (-4.1%)	—
	GRU4Rec	.2811 (-1.5%)	—	—	.1648 (+2.8%)	—
	HSTU	.3097 (+8.6%)	.5754 (+5.1%)	.7716 (+2.5%)	.1720 (+7.3%)	.2606 (+4.3%)
	HSTU-large	.3294 (+15.5%)	.5935 (+8.4%)	.7839 (+4.1%)	.1893 (+18.1%)	.2771 (+10.9%)
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	BERT4Rec	.2816 (-3.4%)	—	—	.1703 (+5.1%)	—
	GRU4Rec	.2813 (-3.2%)	—	—	.1730 (+6.7%)	—
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	.3567 (+22.8%)	.6149 (+11.8%)	.8076 (+5.5%)	.2106 (+30.0%)	.2971 (+17.9%)
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	.0469 (+60.6%)	.1066 (+46.2%)	.1876 (+33.9%)	.0257 (+65.8%)	.0508 (+45.1%)

Table 12. 传统序列推荐器设置 (多遍历、全量打乱) 下各方法在公开数据集上的评估。与 Table 4 相比, 为完整起见, 额外纳入了另外两个基线 (GRU4Rec 和 BERT4Rec)。

指标名称	选择类型		
	贪心	加权	随机
主要交互指标 (NE)	0.495	0.494	0.495
主要消费指标 (NE)	0.792	0.789	0.791

Table 13. 以归一化熵 (NE) 衡量, 随机长度采用不同子序列选择方法时的模型质量比较。

Alpha	最大序列长度							
	1,024		2,048		4,096		8,192	
	稀疏度	s2	稀疏度	s2	稀疏度	s2	稀疏度	s2
1.6	71.5%	89.4%	75.8%	92.3%	79.4%	94.7%	<u>83.8%</u>	<u>97.3%</u>
1.7	<u>57.3%</u>	<u>77.6%</u>	<u>60.6%</u>	<u>79.8%</u>	<u>67.3%</u>	<u>86.6%</u>	<u>74.5%</u>	<u>93.3%</u>
1.8	<u>37.5%</u>	<u>56.2%</u>	<u>42.6%</u>	<u>62.1%</u>	<u>51.9%</u>	<u>74.2%</u>	<u>62.6%</u>	<u>85.5%</u>
1.9	<u>15.0%</u>	<u>25.2%</u>	<u>17.7%</u>	<u>29.0%</u>	<u>29.6%</u>	<u>47.5%</u>	<u>57.8%</u>	<u>80.9%</u>
2.0	<u>1.2%</u>	<u>1.7%</u>	<u>2.5%</u>	<u>3.5%</u>	<u>18.9%</u>	<u>30.8%</u>	<u>57.6%</u>	<u>80.6%</u>

Table 14. 随机长度 (SL) 对 60 天用户交互历史上序列稀疏度的影响。

Alpha	最大序列长度							
	1,024		2,048		4,096		8,192	
	稀疏度	s2	稀疏度	s2	稀疏度	s2	稀疏度	s2
1.6	68.0%	85.0%	74.6%	90.8%	78.6%	93.5%	<u>83.5%</u>	<u>97.3%</u>
1.7	<u>56.3%</u>	<u>76.1%</u>	<u>61.2%</u>	<u>80.6%</u>	<u>67.5%</u>	<u>87.0%</u>	<u>74.3%</u>	<u>93.3%</u>
1.8	<u>38.9%</u>	<u>58.3%</u>	<u>42.0%</u>	<u>61.3%</u>	<u>50.4%</u>	<u>72.4%</u>	<u>61.0%</u>	<u>84.4%</u>
1.9	<u>16.2%</u>	<u>27.3%</u>	<u>17.3%</u>	<u>28.6%</u>	<u>27.2%</u>	<u>44.4%</u>	<u>54.3%</u>	<u>77.8%</u>
2.0	<u>0.9%</u>	<u>1.2%</u>	<u>1.6%</u>	<u>2.1%</u>	<u>13.5%</u>	<u>22.5%</u>	<u>54.0%</u>	<u>77.4%</u>

Table 15. 随机长度 (SL) 对 90 天用户交互历史上序列稀疏度的影响。

评估策略	相较完整序列基线的平均 NE 差异		
	模型类型	2048 / 52% 稀疏度	4096 / 75% 稀疏度
零样本	HSTU (Raffel et al., 2020)	6.46%	10.35%
	HSTU-RoPE (Peng et al., 2024)	7.51%	11.27%
微调	HSTU (Raffel et al., 2020)	1.92%	2.21%
	HSTU-RoPE (Peng et al., 2024)	1.61%	2.19%
随机长度 (SL)	HSTU	0.098%	0.64%

Table 16. 随机长度 (SL) 与现有长度外推方法的比较。